

DEGREE PROJECT FOR MASTER OF COMPUTER SCIENCE WITH
SPECIALIZATION IN CYBERSECURITY DEPARTMENT OF ENGINEERING
SCIENCE UNIVERSITY WEST

Comparative Evaluation of Open-Source SOAR Tools with FortiGate Integration Using Real-World Honeypot Attack Data

Alphonse JOSEPH
Filmon Mehari GEBREZGHI



A THESIS SUBMITTED TO THE DEPARTMENT OF ENGINEERING SCIENCE
IN PARTIAL FULFILMENT OF THE REQUIREMENTS FOR THE DEGREE OF
MASTER OF COMPUTER SCIENCE WITH SPECIALIZATION IN CYBERSECURITY
AT UNIVERSITY WEST
May 2026

Author(s):	Alphonse Joseph Filmon Mehari Gebrezghi
Examiner:	Vira Shendryk
Supervisor:	Robert Andersson, Högskolan Väst
Program:	Master Programme in Cybersecurity
Main field of study:	Computer Science
Credits:	60 Higher Education credits
Keywords:	SOAR, Shuffle, Wazuh, TheHive, Cortex, FortiGate, Honeypot, Cybersecurity, Incident Response, Automation, Threat Intelligence, False Positive Rate, Docker, Active Response
Template:	University West, IV-Master 2.10
Publisher:	University West, Department of Engineering Science S-461 86 Trollhättan, SWEDEN Phone: + 46 520 22 30 00 Fax: + 46 520 22 32 99 Web: www.hv.se

Summary

Security Operations Centers increasingly depend on Security Orchestration, Automation and Response (SOAR) platforms to manage the growing volume of cybersecurity alerts. This thesis presents a comparative empirical evaluation of two open-source SOAR approaches: Shuffle, a dedicated visual SOAR orchestrator, and a Wazuh + Cortex + TheHive integrated stack.

Both platforms were deployed on identical hardware and received the same live attack data simultaneously sourced from a T-Pot 24.04 multi-honeypot system (exposing seven sensors to the public internet) and a FortiGate 50G next-generation firewall. Four identical automation playbooks were implemented on each platform: IP address blocking via the FortiGate REST API, alert enrichment via AbuseIPDB, alert notification in TheHive, and case creation in TheHive.

The platforms were evaluated across six metrics: playbook success rate, response latency, false positive rate, integration effort, average CPU usage, and average RAM usage. The results show that Shuffle offers lower integration effort and faster playbook construction through its visual drag-and-drop interface, while the Wazuh stack provides superior detection depth, richer alert context, and tighter integration between detection, enrichment, and case management. Resource consumption is significantly higher for the Wazuh stack due to Cassandra, Elasticsearch, and TheHive running concurrently. Practical recommendations are provided for organizations that select between these approaches based on team size, volume of alerts, and existing infrastructure. Wazuh demonstrated 6.3 times lower average response latency (0.387 seconds versus 2.5 seconds for Shuffle), while both platforms achieved 100% playbook success rate. Shuffle required fewer lines of configuration code (141 versus 195) and offers a visual drag-and-drop interface that lowers the barrier to automation for teams without scripting expertise.

Preface

This thesis was conducted as part of the Master of Computer Science with Specialization in Cybersecurity programme at University West, Trollhättan, Sweden, during the academic year 2025–2026.

We would like to thank supervisor Robert Andersson for guidance throughout the project, and examiner Vira Shendryk for valuable feedback. Special thanks to the University West IT department for providing access to the physical testbed hardware used in this study.

The T-Pot honeypot platform was developed by Deutsche Telekom Security GmbH and is used here purely for academic research purposes. All attack data collected originated from public internet scanners interacting with our honeypot sensors; no systems were attacked without authorisation.

Filmon Mehari Gebrezghi
Alphonse Joseph
Trollhättan, May 2026

Affirmation

This master degree report, *Comparative Evaluation of Open-Source SOAR Tools with FortiGate Integration*, was written as part of the master degree work required to obtain a Master of Science with specialization in Cybersecurity at University West. All material in this report that is not our own is clearly identified and used appropriately. The main part of the work included in this degree project has not previously been published or used for obtaining another degree.

Filmon Mehari Gebrezghi

Date: May 2026

Alphonse Joseph

Date: May 2026

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Aim	2
1.3	Research Questions	2
1.4	Contributions	3
1.5	Scope and Limitations	3
1.6	Thesis Structure	3
2	Background and Related Work	5
2.1	Background	5
2.1.1	Security Orchestration, Automation, and Response (SOAR)	5
2.1.2	Shuffle	6
2.1.3	Wazuh, Cortex and TheHive	6
2.1.4	Honeypot Systems	6
2.1.5	Next-Generation Firewalls and FortiGate	7
2.1.6	Evaluation Metrics for SOAR Platforms	7
2.2	Related Work	8
2.2.1	SOAR Platform Evaluations	8
2.2.2	SOAR and Honeypot Integration	8
2.2.3	Open-Source Security Tool Comparisons	8
2.2.4	Honeypot and Firewall Integration	9
2.2.5	Research Gap Identification	9
3	Research Methodology	10
3.1	Research Design	10
3.2	Testbed Architecture	11
3.2.1	Hardware and Network Configuration	11
3.2.2	Network Topology	11
3.2.3	Hardware Setup	12
3.2.4	T-Pot Honeypot Configuration	12
3.2.5	FortiGate 50G Configuration	13
3.2.6	SOAR Platform Deployment	13
3.2.7	Playbook Design	13
3.3	Data Gathering Methods	14
3.3.1	Live Exposure Period	14

3.3.2	Controlled Attack Scenarios	15
3.3.3	Alert Sampling and Labeling	15
3.3.4	Integration Effort Measurement	16
3.3.5	Resource Monitoring	16
3.4	Data Analysis Methods	16
3.5	Validity Considerations	18
3.6	Ethical Considerations	19
4	Testbed Implementation and SOAR Integration	20
4.1	System Overview	20
4.2	Existing Component Deployment	21
4.3	Custom Integration Components	22
4.4	Playbook Implementation	22
4.5	Implementation Challenges	23
4.6	Port Allocation	23
5	Results	24
5.1	Data Collection Summary	24
5.2	Threat Intelligence Analysis External Attacker IPs	26
5.3	Response Latency	28
5.4	Playbook Success Rate	29
5.4.1	Shuffle Execution Statistics	30
5.5	False Positive Rate and Alert Classification	30
5.5.1	Classification Results	30
5.6	Integration Effort	32
5.7	Resource Consumption	33
5.8	Results Summary	35
6	Discussion	36
6.1	Interpretation of Response Latency Findings	36
6.2	Interpretation of False Positive Rate Findings	36
6.3	Interpretation of Integration Effort Findings	37
6.4	Interpretation of Resource Consumption Findings	37
6.5	Interpretation of Threat Intelligence Findings	37
6.6	Interpretation of SOAR Infrastructure Scanning Findings	38
6.7	Comparison with Related Work	38
7	Conclusion	39
7.1	Achievements	39
7.1.1	Comparative Performance Findings	39
7.1.2	Threat Intelligence and Infrastructure Findings	40
7.1.3	Practical Recommendations	40
7.2	Future Work and Research	41
7.2.1	Larger Alert Sample and Extended Collection Period	41
7.2.2	Multi-Site and Cloud-Native Deployments	41

7.2.3	Automated Ground-Truth Labeling	41
7.2.4	Extended Platform Coverage	41
7.2.5	Performance Optimization for Container-Based Orchestration	42
7.3	Critical Discussion	42
7.4	Generalization of the Result	43
Bibliography		44

1 | Introduction

1.1 Motivation

Network systems exposed to the public internet are constantly scanned by automated tools through common service ports. Attackers often focus on Secure Shell (SSH) by trying many password combinations until they find one that works. They use the Hypertext Transfer Protocol (HTTP) to take advantage of weaknesses in web applications and rely on the Server Message Block (SMB) protocol to move across the network and install malicious software. Security operations centers (SOCs) find it difficult to handle the large number of alerts that these attacks create, which can overwhelm analysts, slow the response to incidents, and increase risk for the organization. The amount of activity makes manual review and prioritization especially challenging for organizations with small security teams or limited budgets.

Security Orchestration, Automation, and Response (SOAR) platforms automate incident handling processes and thereby help address this challenge. SOAR tools connect different security systems, improve alerts with more information, and use ready-made playbooks to automatically handle threats. A single incoming alert can trigger IP reputation lookups, firewall rule updates, analyst notifications, and case ticket creation without human intervention at each step [1].

Commercial SOAR solutions such as Palo Alto Cortex XSOAR, Splunk Phantom, and IBM Resilient have many advanced features, but carry licensing costs that exceed the budgets of educational institutions and small enterprises [2]. This cost barrier motivates interest in open-source alternatives. The open-source SOAR ecosystem has matured considerably in recent years, making these platforms viable for production use.

Open source solutions have become viable and more cost-effective alternatives. **Shuffle** provides a visual workflow editor and a library of pre-built application connectors, allowing rapid playbook construction without programming expertise [3]. The combination of **Wazuh** for host and network detection, **Cortex** for indicator enrichment through modular analyzers, and **TheHive** for structured case management represents a more component-rich alternative that draws on separate mature open-source projects [4, 5]. The diversity of these architectures from all-in-one visual orchestrators to modular stacks makes platform selection a non-trivial decision.

FortiGate next-generation firewalls give security teams a clear view of network activity and help block threats before they cause damage. FortiGate 50G performs Deep Packet Inspection (DPI) to identify application-layer threats including SQL injection, cross-site scripting, and malware command-and-control traffic. In addition, these firewalls can send

threat notifications via syslog and connect to REST APIs, allowing automated defenses to be activated without human intervention [6]. FortiGate devices are among the most widely deployed enterprise firewalls globally, adding practical industrial relevance to this study.

Honeypot technology provides an ethical method to collect real attack data by simulating vulnerable systems. T-Pot, a multi-purpose honeypot platform developed by Deutsche Telekom, runs several honeypot sensors in a single environment, including Cowrie for SSH, Dionaea for malware, Suricata for intrusion detection, and additional sensors for web attacks, HTTPS traffic, and ICS/SCADA emulation.[7, 8].

This provides comprehensive attack surface visibility and captures authentic, uncontrolled attack data from real internet adversaries port scanners, credential stuffers, malware distributors, and CS/SCADA probers, which is substantially more representative than synthetic traffic generators.

Despite the availability of open-source SOAR platforms, firewalls, and honeypots, very few empirical studies evaluate them under identical conditions using real attack data from both a multi-sensor honeypot deployment and a physical next-generation firewall simultaneously [9]. Existing research focuses on single tools or theoretical comparisons, leaving organizations without data-driven guidance for selecting open-source SOAR solutions.

1.2 Aim

The primary aim of this thesis project is to conduct an empirical comparative analysis of two open-source SOAR approaches: **Shuffle** (dedicated orchestrator) and the **Wazuh + Cortex + TheHive** stack (integrated detection and response platform). Using identical real-world attack data collected from two sensors : T-Pot honeypot and a FortiGate 50G firewall. This study evaluates and compares their automated response capabilities, operational characteristics, and practical suitability for organizations with limited security resources. The findings will provide evidence-based recommendations to assist security practitioners in making informed decisions about the selection of SOAR tools and contribute to academic understanding of the effectiveness of open-source security automation.

1.3 Research Questions

To achieve the stated goal, this project addresses the following research questions.

1. How do open-source SOAR solutions (Shuffle and the Wazuh + Cortex + TheHive stack) compare in automating incident response to real-world attacks captured from T-Pot honeypots and FortiGate firewall logs?
2. How do the platforms differ in terms of response latency, resource efficiency, integration effort when processing identical alerts?

1.4 Contributions

This thesis project makes several key contributions to both academic knowledge and practical security operations:-

- **Empirical Comparison Framework:** Development of a reproducible methodology for comparing open-source SOAR platforms using real attack data from both honeypot and firewall sources.
- **Evidence-Based Evaluation:** Comprehensive analysis of two widely used open-source SOAR approaches based on multiple operational metrics, including playbook success rate, response latency, false-positive handling, integration effort, and resource consumption.
- **Practical Guidance:** Actionable recommendations for security practitioners in resource-constrained organizations, helping them select appropriate open-source security automation solutions based on empirical evidence.
- **Academic Knowledge Contribution:** Addressing the identified gap in the literature regarding empirical evaluations of open-source SOAR tools using real-world attack data from both host and network level sensors.
- **Publicly Accessible Artifacts:** The deployment scripts, configurations, and experimental procedures developed for this project are publicly available at <https://github.com/Nyx-Sentinel/Comparative-Evaluation-of-Open-Source-SOAR-Tools-with-FortiGate-and-Honeypot-Integration> to support reproducibility and future research

1.5 Scope and Limitations

The evaluation is conducted on a single physical testbed within the University West network environment. The results reflect the behavior of each platform under the specific hardware, network conditions, and attack distribution encountered during the collection period. Generalization to other environments, hardware configurations, or attack populations requires further study.

The validation sample of 100 alerts per platform provides a basis for comparison but is not statistically sufficient to establish population-level precision estimates. Manual alert labeling introduces subjectivity that the inter-rater consensus process mitigates but cannot entirely eliminate. A detailed discussion of limitations is presented in Chapter 6.

1.6 Thesis Structure

Chapter 2 reviews related work on SOAR systems, honeypot deployments, and firewall integration. Chapter 3 describes the research methodology, testbed design, and evaluation metrics. Chapter 4 documents the technical implementation of the testbed and both SOAR platforms. Chapter 5 presents the measured results. Chapter 6 discusses the findings in

relation to the existing literature. Chapter 7 concludes the thesis and outlines the directions for future work.

2 | Background and Related Work

This chapter provides the theoretical foundation and contextual understanding necessary for this study. The first section introduces key concepts, technologies, and methodologies relevant to security orchestration, automation and response (SOAR), network based threat detection and honeypot-based attack data collection. The second section reviews the existing scientific literature and related research in the field of SOAR evaluation, honeypot-SOAR integration, and open-source security automation.

2.1 Background

2.1.1 Security Orchestration, Automation, and Response (SOAR)

Security Orchestration, Automation, and Response (SOAR) platforms have become essential components of modern cybersecurity operations. Introduced by Gartner in 2017, the term describes systems that unify three core capabilities: orchestration of disparate security tools, automation of repetitive workflows, and structured incident response management [1].

A SOAR platform serves three interconnected functions. **Orchestration** connects disparate security tools, firewalls, intrusion detection systems, threat intelligence feeds, and ticketing systems through a common integration layer. **Automation** encodes analyst decision logic into playbooks that execute response actions without requiring manual intervention for each alert. **Response** provides structured workflows to contain threats, enrich indicators, and document case outcomes [10].

The operational motivation for SOAR adoption comes from alert fatigue in security operations centers. Analysts responsible for triaging high volumes of daily alerts experience diminished response quality as event rates increase. Automating high-confidence, low-complexity responses such as blocking a known-malicious IP address or querying a reputation database preserves the attention of the analyst for events requiring human judgment [2].

SOAR platforms typically operate on alerts generated by external detection systems, applying workflows that determine appropriate responses based on predefined rules or analytical outputs. However, architectural approaches vary: some platforms function primarily as orchestration layers, while others integrate detection, enrichment, and response components within a unified ecosystem.

In this study, two distinct open-source approaches are examined. Shuffle represents a dedicated orchestration platform that emphasizes visual workflow design and rapid automation deployment. In contrast, the Wazuh + Cortex + TheHive stack combines host-based

detection, analytical enrichment, and case management into a distributed SOAR-like architecture.

2.1.2 Shuffle

Shuffle is an open-source SOAR platform designed specifically for security orchestration and automation, released under the GNU General Public License [11]. Its architecture is based on a visual drag-and-drop workflow editor that connects the application nodes representing external services. Triggers include webhooks, scheduled intervals, and email ingestion. Application connectors are implemented as Docker containers, allowing community-contributed integrations to run in isolation from the core platform. Shuffle is designed to minimize the configuration overhead required to build functional playbooks, making it accessible to teams without dedicated development resources [11].

2.1.3 Wazuh, Cortex and TheHive

Wazuh is an open-source security monitoring platform that extends the OSSEC host-based intrusion detection system (HIDS) with centralized log collection, file integrity monitoring, vulnerability detection, and active response capabilities [12]. Based on the Elastic Stack, Wazuh uses a centralized manager with agents deployed on monitored endpoints. Wazuh processes events against a rule engine and produces structured alerts that can be forwarded to downstream systems. The active response module enables automated execution of scripts in response to matching alerts, forming the response layer of a SOAR workflow when combined with enrichment and case management components.

Cortex functions as a modular observable analysis engine [4]. Analyzers are independent programs that accept indicators IP addresses, domain names, file hashes, and URLs, and return structured enrichment reports from external data sources, including threat intelligence feeds and reputation databases. Cortex decouples enrichment logic from the case management layer, allowing analyzers to be added or updated without modifying the platform configuration.

TheHive is an open-source, scalable security incident response platform designed for security operations teams [5]. The cases in TheHive correspond to security incidents and contain observables, tasks, timelines, and analyst notes. TheHive integrates with Cortex to trigger analyzer execution directly from the case interface and with Wazuh to receive alerts as automated case creation events. The combination of Wazuh, Cortex, and TheHive forms a complete detection, enrichment, and case management pipeline that covers the core functions of a SOAR platform.

2.1.4 Honeypot Systems

A honeypot is a networked system deliberately configured to simulate vulnerable services and attract unauthorized connection attempts. Because any incoming connection represents unsolicited activity, honeypots generate high-fidelity attack data without the privacy complications of monitoring production traffic [9].

T-Pot is a multi-purpose honeypot platform developed and maintained by Deutsche Telekom Security [7, 8]. A single T-Pot installation deploys multiple independent honeypot sensors as Docker containers, each emulating a different service or protocol. Key sensors include Cowrie (SSH/Telnet), Dionaea (SMB/FTP / HTTP malware capture), Honeytrap (network probes), Tanner (web applications), Suricata (intrusion detection), ConPot (ICS/SCADA emulation) and H0neytr4p (HTTPS traffic). All sensor output is ingested by an embedded ELK stack and visualized through a Kibana dashboard.

2.1.5 Next-Generation Firewalls and FortiGate

Next-Generation Firewalls (NGFWs) extend traditional statefull packet inspection with application-layer visibility, user identity awareness, intrusion prevention, and SSL deep packet inspection. Unlike statefull inspection firewalls that filter based solely on ports and protocols, NGFWs perform Deep Packet Inspection (DPI) to identify application-layer threats including SQL injection, cross-site scripting, and malware command-and-control traffic. These capabilities generate security events on the network perimeter that complement host-based and honeypot-sourced telemetry in a layered detection architecture [6].

The FortiGate 50G firewall used in this study provides three capabilities relevant to SOAR integration: syslog export for streaming security events to external log collectors, a REST API for programmatic configuration and automated response actions, and an Intrusion Prevention System for real-time attack detection [13]. The REST API capability enables SOAR platforms to implement automated IP blocking responses directly at the enforcement point.

2.1.6 Evaluation Metrics for SOAR Platforms

Evaluating SOAR platforms requires multi-dimensional assessment beyond simple detection rates. The six metrics used in this study are:

- **Playbook success rate:** Successful executions divided by total triggered executions, expressed as a percentage.
- **Response latency:** Time from alert ingestion to final completion of the action of the playbook, measured in seconds [14].
- **False positive rate:** Proportion of playbooks triggered by benign events, indicating wasted automation effort [2].
- **Integration effort:** Person-hours required to configure connectors, build playbooks, and establish data flows [15].
- **CPU utilization:** Processor usage during sustained operation [16].
- **RAM consumption:** Memory usage during sustained operation [16].

These metrics form the basis of the evaluation methodology presented in Chapter 3.

2.2 Related Work

2.2.1 SOAR Platform Evaluations

Academic research on SOAR platform evaluation remains limited, and most published work focuses on commercial solutions or theoretical frameworks.

Bridges et al. (2023) conducted a hands-on user study testing six commercial SOAR tools, measuring efficiency, configuration complexity, and response metrics through user studies in a cyber range environment [10]. Their work established methodologies for evaluating SOAR usability and performance.

Norem et al. (2021) developed a mathematical framework to evaluate SOAR tools, applying structured assessment methodologies to measure operator performance and workflow effectiveness [16]. Sworna et al. (2023) created APIRO, a framework for automated security tools API recommendation, with experimental validation across three SOAR-related tools [15].

2.2.2 SOAR and Honeypot Integration

Several studies have explored the integration of SOAR capabilities with honeypot systems for automated threat mitigation.

Kumar et al.[4] demonstrated the integration of a Cowrie SSH honeypot with a SOAR platform to automate IP blocking of brute-force sources, reporting measurable reductions in repeat attack attempts following automated response activation. Their architecture relied on a single honeypot sensor and did not incorporate a hardware firewall as a data source or response target within the experimental design [4].

Kumar, Gupta, and Singh (2022) developed a Security Orchestration, Automation, and Response Engine specifically for deployment of behavioral honeypots, establishing that automated orchestration reduces analyst response time for honeypot-sourced alerts [17]. The evaluation used controlled attack scenarios rather than passive live internet exposure, limiting the representativeness of the attack distribution.

Aljahdali and Alsulami (2025) implemented an automated SOC environment using SIEM and SOAR on a virtual testbed with simulated attacks [14].

2.2.3 Open-Source Security Tool Comparisons

Manzoor et al. (2024) evaluated six open-source SIEM tools, including Wazuh, for security and performance characteristics [2]. Their findings regarding Wazuh's detection capabilities inform this study's baseline expectations for the Wazuh+Cortex+TheHive stack.

Empl et al. (2022) proposed and implemented SOAR4IoT, a SOAR framework for securing IoT assets using digital twins, with prototypical validation and tests with sensors [5]. Nyre-Yu (2021) conducted a qualitative study identifying expertise gaps in cyber incident response through analysis of nine SOAR platforms [18].

2.2.4 Honeypot and Firewall Integration

Fischer, Smith, and Golling (2022) analysed attack patterns collected from a high-interaction honeypot over an extended exposure period, characterising the geographic distribution, timing, and protocol preferences of automated scanners and targeted human actors [9]. Their work informs the expectation of attack volumes and types in the present study but does not evaluate SOAR response.

Chen, Wang, and Lee (2025) explored the integration of AI-infused firewalls with automated threat detection systems, demonstrating that machine learning-enhanced firewalls can reduce false positive rates while maintaining detection accuracy for application-layer attacks such as SQL injection and cross-site scripting [6]. Their findings support the use of next-generation firewalls like FortiGate as active response platforms within SOAR workflows, rather than passive data sources only.

Sharma, Gupta, and Kumar (2023) proposed HoneyTrack, an improved honeypot architecture designed specifically to capture attacker behaviour in industrial control system environments [3]. Unlike traditional honeypots that focus on IT protocols such as SSH and HTTP, HoneyTrack emulates industrial protocols including Modbus and IEC 104. This is relevant to the present study because T-Pot includes ConPot, an ICS/SCADA emulation sensor that generates attack data of the type HoneyTrack was designed to capture.

Suhendi et al. (2023) evaluated the integration of the Artillery honeypot with Wazuh SIEM, demonstrating that combining honeypot-sourced alerts with Wazuh's rule-based analysis improves detection coverage for SSH brute-force attempts and port scanning activities [12]. Their work validates the architectural choice in the present study of using Wazuh as the detection and alerting engine while receiving attack data from multiple honeypot sensors simultaneously.

2.2.5 Research Gap Identification

The reviewed literature establishes the individual utility of SOAR platforms, honeypot deployments, and next-generation firewalls. However, based on the literature reviewed in this chapter, several aspects appear to be underrepresented or not addressed in previous studies. These include evaluating Shuffle and the Wazuh stack against each other using identical input data; incorporating a physical firewall as both a passive data source and an active automated response target; combining live internet-exposed honeypot data with hardware firewall alert streams as simultaneous input to both platforms; measuring all six performance dimensions (playbook success rate, response latency, false positive rate, integration effort, CPU usage, and RAM usage) within a single comparative study; and conducting the evaluation on dedicated physical hardware rather than virtual or cloud-hosted infrastructure.

The present study addresses these gaps through a controlled physical testbed experiment using live attack data collected over an extended period of exposure to the internet.

3 | Research Methodology

3.1 Research Design

This study employs a controlled experimental design to compare two open-source SOAR configurations, Shuffle (dedicated workflow orchestrator) and the Wazuh + Cortex + TheHive stack (integrated detection, enrichment and case management), under identical real-world conditions. The experimental approach is appropriate because the research questions call for quantitative performance measurement across defined metrics rather than exploratory or interpretive analysis. A single physical testbed provides the controlled environment in which both platforms receive the same input data, execute equivalent playbooks, and are evaluated on identical tasks.

The design follows a comparative structure within the testbed: both platforms operate concurrently on the same physical host (Machine 2, IP 193.10.203.151), receiving identical log streams split from a single Logstash pipeline. This architecture eliminates network path differences, hardware performance variation, and input data differences as confounding factors in the comparison.

The study is empirical in nature. Attack data is collected from real internet traffic rather than synthetic scenarios, ensuring that the evaluated workloads reflect genuine attacker behavior rather than laboratory artifacts. From March 13 to April 15, 2026, the T-Pot honeypot (Machine 1) was continuously exposed to the public internet. Controlled attack injections from a dedicated Kali Linux laptop supplement the passive collection with known ground-truth events.

The dual-source design captures comprehensive incident response coverage. T-Pot captures host-based attacks (SSH brute-force, SMB exploitation, HTTP web attacks) while the FortiGate 50G firewall captures network-based threats (SQL injection, cross-site scripting, port scans, IPS alerts). Using both sources ensures the comparison covers the full range of attack types encountered in enterprise environments.

Six quantitative metrics are used to compare the platforms: playbook success rate, response latency, false positive rate, integration effort, CPU utilization, and RAM consumption. These metrics are defined in Section 3.3 and operationalized through direct measurement from platform logs, system monitoring, and time-tracked configuration logs.

3.2 Testbed Architecture

3.2.1 Hardware and Network Configuration

The testbed comprises three physical components connected to the university network. Table 3.1 summarizes the hardware configuration and addressing.

Table 3.1: Hardware and Network Configuration

Component	Role	IP Address	Specification
Machine 1 (T-Pot)	Honeypot data source	193.10.203.150	T-Pot 24.04, Ubuntu 24.04
Machine 2 (SOAR)	SOAR platforms host	193.10.203.151	Ubuntu 22.04, 15GB RAM
FortiGate 50G	NGFW, data source, response target	193.10.203.159	FortiOS v7.6.6
Kali Linux laptop	Controlled attack generation	LAN-connected	Kali Linux 2025.4

Static IP addresses were configured on both Linux machines using Netplan to prevent DHCP-caused address changes that would invalidate all configured integrations. The FortiGate trusted host setting was configured using the subnet 193.10.203.128/25 rather than individual IP addresses, ensuring API access remained functional even in the event of minor network changes.

3.2.2 Network Topology

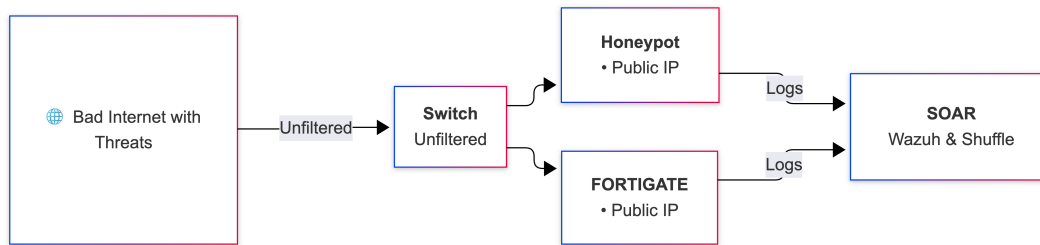


Figure 3.1: Physical testbed network topology.

As illustrated in Figure 3.1, the university provides public IP addresses in the 193.10.203.0/25 subnet to both physical machines and the FortiGate WAN interface. An unfiltered switch aggregates the university uplink and distributes connectivity to Machine 1 and the FortiGate WAN port, giving both direct exposure to internet traffic without upstream filtering. The FortiGate LAN interface connects to Machine 2, placing the SOAR host behind firewall policy enforcement.

The purpose of this dual-path design is to expose Machine 1 to unfiltered internet traffic for maximum attack capture while allowing the FortiGate to serve as both a data source (IPS alerts) and an automated response target (IP blocking via REST API). Both SOAR platforms receive identical log streams through a single Logstash pipeline on Machine 2, which accepts Beats input on port 5044 and syslog input on port 514, then splits all incoming events into

two output streams: an HTTP output posting to Shuffle’s webhook endpoint and a TCP output forwarding to Wazuh’s agent listener on port 1514.

3.2.3 Hardware Setup

The testbed comprises two physical desktop computers allocated by University West. Both machines run Ubuntu Server 22.04 LTS without a graphical environment. A FortiGate 50G next-generation firewall and a managed Cisco network switch complete the hardware setup. Table 3.1 in Section 3.2.1 lists the hardware specifications and IP addresses.

3.2.4 T-Pot Honeypot Configuration

T-Pot version 24.04.1 Standard edition is deployed on Machine 1 using the official installation script from the Deutsche Telekom Security repository. The installer automatically provisions Docker Engine and all sensor containers without manual dependency management. Following installation, the host reboots with T-Pot’s `iptables` rules active, exposing honeypot sensor ports to the internet. Management SSH access is relocated to port 64295 and the web dashboard to port 64297. Seven active honeypot sensors are operational on Machine 1, as listed in Table 3.2. Each sensor listens on specific ports that emulate vulnerable network services.

Table 3.2: Active T-Pot Honeypot Sensors and Port Mapping

Sensor	Protocol / Ports	Attack Category
Cowrie	SSH, Telnet (22, 23)	Credential brute-force, shell interaction
Dionaea	SMB, FTP, HTTP, SQL (20, 21, 42, 81, 135, 445, 1433, 1723, 1883, 3306, 27017)	Malware delivery, exploitation
Suricata	All traffic (IDS)	Signature-based intrusion detection
Honeytrap	All ports	Network reconnaissance, scanning
Tanner	HTTP (80)	Web application attacks
H0neytr4p	HTTPS (443)	Encrypted traffic capture
ConPot	SNMP (161), IEC 104 (2404), IPMI (623)	ICS/SCADA emulation

The internal T-pot logstash pipeline processes all sensor logs and forwards them to two destinations: the embedded Elasticsearch/Kibana stack for local visualization, and the central Logstash on Machine 2 via a configured HTTP output. Both outputs receive identical event data simultaneously, ensuring no data loss on either path.

3.2.5 FortiGate 50G Configuration

The FortiGate 50G is configured with the following active capabilities throughout the data collection period [13].

- Intrusion Prevention System (IPS) for detecting network-based attacks
- Deep Packet Inspection (DPI) for application-layer threat identification
- Syslog forwarding to the Logstash pipeline on Machine 2 (UDP port 514)
- REST API access on port 443 for automated response actions

The FortiGate is positioned with its WAN interface connected to the unfiltered switch (direct internet exposure) and its LAN interface connected to the Machine 2. This topology places the SOAR machine behind FortiGate policy enforcement while allowing the honeypot to receive unfiltered traffic directly.

3.2.6 SOAR Platform Deployment

Both SOAR platforms are deployed on Machine 2. To prevent port conflicts between the two platforms, careful port allocation was required. Table 4.2 presents the final port assignment. The Wazuh indexer uses port 9201 instead of the default 9200 to avoid conflict with Shuffle's OpenSearch container. Detailed port allocation is documented in Chapter 4. Shuffle was configured with a webhook endpoint to receive forwarded alerts from the Wazuh integration pipeline. Wazuh manager listens for agent connections on port 1514. The Logstash pipeline on Machine 2 listens for Beats input on port 5044 (receiving T-Pot logs) and syslog input on port 514 (receiving FortiGate logs).

3.2.7 Playbook Design

Four playbooks are implemented on both platforms using functionally equivalent trigger conditions and action sequences, ensuring that the comparison reflects platform capability rather than differences in workflow logic. Table 3.3 summarises the four playbooks and their purpose.

Table 3.3: Playbook Overview and Purpose

ID	Name	Purpose
PB1 / WZ-PB1	Automated IP Blocking	Block malicious source IP at the FortiGate firewall
PB2 / WZ-PB2	Threat Intelligence Enrichment	Query AbuseIPDB for IP reputation and threat context
PB3 / WZ-PB3	Alert Notification	Create structured alert in TheHive for analyst awareness
PB4 / WZ-PB4	Case Creation	Create full investigation case in TheHive for high-severity incidents

Playbook 1: Automated IP Blocking

Trigger: Detection of a suspicious or malicious source IP address from T-Pot honeypot data or FortiGate IPS alerts.

Action: The playbook extracts the source IP address from the alert payload, queries the AbuseIPDB threat intelligence API to retrieve the IP's abuse confidence score, and if the score exceeds 75%, sends an authenticated REST API request to the FortiGate 50G to create a deny policy entry for that address. The blocking action is logged with a timestamp and the triggering alert identifier.

Implementation difference: In Shuffle, this workflow is implemented as a three node chain: Webhook trigger → HTTP enrichment node → HTTP FortiGate API node. On the Wazuh stack, an equivalent active response script executes on the Wazuh manager and forwards the block instruction to the FortiGate via the REST API.

Playbook 2: Threat Intelligence Enrichment (PB2 / WZ-PB2)

Trigger: Any alert containing a source IP address. **Action:** The playbook submits the source IP address to AbuseIPDB to retrieve the IP's abuse confidence score, country of origin, Internet Service Provider (ISP), and historical report count. The returned data is appended to the alert record as structured metadata.

Playbook 3: Alert Notification (PB3 / WZ-PB3)

Trigger: Any security alert requiring analyst awareness. **Action:** The playbook creates a structured alert in TheHive, making it visible to SOC analysts in the incident management interface. The alert includes the source IP, alert description, timestamp, severity level, and relevant tags.

Playbook 4: Case Creation (PB4 / WZ-PB4)

Trigger: Alerts classified as high-priority security incidents (e.g., abuse confidence score above 75% or IPS critical severity). **Action:** The playbook creates a full investigation case in TheHive with severity classification, tags, assignment, and a structured description containing all available evidence (source IP, attack type, enrichment results, and blocking status).

3.3 Data Gathering Methods

The data collection process is divided into three phases: live internet exposure for passive attack capture, controlled attack injection for ground-truth labeling, and resource monitoring for performance measurement.

3.3.1 Live Exposure Period

Machine 1 is connected to the unfiltered switch with its public IP address for a minimum of fourteen consecutive days. All incoming unsolicited connections are logged automatically by

the respective honeypot sensor. No pre-filtering or attack-type selection is applied during the passive collection phase. The fourteen-day window follows the duration used in comparable honeypot studies [9]. During this period, the FortiGate 50G simultaneously logs network-based threats detected by its IPS engine. All FortiGate security events are forwarded via syslog to the central Logstash pipeline on Machine 2.

3.3.2 Controlled Attack Scenarios

Controlled attacks are injected from a Kali Linux laptop connected to the same network switch as Machine 1. These injections provide ground-truth events whose attack type, source, and timing are known in advance, supporting the manual labeling validation process. All controlled attacks target Machine 1 exclusively. Table 3.4 lists the attack scenarios executed.

Table 3.4: Planned Controlled Attack Scenarios

Attack Type	Tool	Target Sensor	Day
Network port scan	Nmap	Honeytrap, Suricata	Day 2
SSH brute-force	Hydra	Cowrie	Day 4
SMB exploitation	Metasploit	Dionaea	Day 6
HTTP injection	SQLmap, Nikto	Tanner	Day 8
FortiGate IPS test	curl, manual	FortiGate IPS	Day 10

3.3.3 Alert Sampling and Labeling

Following the collection period, a random sample of alerts is drawn from each platform’s execution log using a Python random sampling script. Random sampling eliminates the selection bias of manual curation. The target sample size is 100 alerts per platform, which balances statistical relevance with the practical constraints of manual labeling. This sample size aligns with comparable security tool evaluation studies [10]. Each sampled alert is labeled independently by both researchers according to the following scheme:

- **True Positive (TP):** The alert corresponds to a genuine attack event, and the playbook executed all intended actions correctly.
- **False Positive (FP):** The alert was triggered by benign or ambiguous traffic and the playbook execution was unwarranted.
- **Gray:** The nature of the triggering event is ambiguous and requires contextual judgment to classify.

Gray cases are resolved through structured discussion between both researchers until consensus is reached. Cohen’s Kappa coefficient is calculated to quantify inter-rater reliability.

3.3.4 Integration Effort Measurement

Integration effort is recorded independently by both researchers during the platform setup phase using a structured time-logging protocol. Each researcher maintains a timestamped activity log that notes the start and end time of each configuration task. The logged categories are: initial platform installation, dependency conflict resolution, API connector configuration, playbook development for each of the four workflows, and end-to-end verification and testing.

To supplement the time-based measurement, the number of configuration steps required per platform is also recorded. A configuration step is defined as any distinct action that requires either a command-line instruction, a modification to a configuration file, or a manual interaction with a web interface.

The total integration effort reported for each platform is the mean of the two independently recorded hour totals. Any discrepancy exceeding 20% between the two researchers' recorded times is resolved through a reconciliation session.

3.3.5 Resource Monitoring

CPU and RAM consumption on Machine 2 are recorded at regular intervals using a shell script that invokes the `top` and `free` utilities in batch mode. Two monitoring sessions are conducted: a steady-state session with measurements every five minutes over 24 hours on collection Day 7, and a mass-attack session with measurements every minute during a 30-minute Hydra SSH brute-force attack on Day 4. Three resource metrics are extracted from each session: mean CPU utilization, peak CPU utilization, and peak RAM consumption in gigabytes.

3.4 Data Analysis Methods

Six quantitative metrics form the basis of the comparative analysis. table 3.5 defines each metric and its measurement method.

Table 3.5: Evaluation Metrics Definition and Measurement Method

Metric	Unit	Measurement Method
Playbook success rate	Percentage	Successful executions divided by total triggered executions, multiplied by 100. Extracted from Shuffle execution history logs and Wazuh active response logs.
Response latency	Seconds	Time difference between alert ingestion timestamp and final playbook action completion timestamp. For Shuffle: from execution logs. For Wazuh: from alert creation to TheHive case creation.
False positive rate	Percentage	False positive alerts divided by total sampled alerts (TP + FP + Gray), multiplied by 100. Derived from manual labeling of 100 randomly sampled alerts per platform.
Integration effort	Person-hours	Mean of independently recorded time logs from both researchers, covering installation, configuration, API integration, playbook development, and debugging. Supplemented by configuration step counts.
CPU utilization	Percentage	Mean and peak CPU usage recorded via <code>top</code> command during steady-state (24 hours, 5-minute intervals) and mass-attack (30 minutes, 1-minute intervals) monitoring sessions.
RAM consumption	Gigabytes	Mean and peak memory usage recorded via <code>free</code> command during the same monitoring sessions.

Playbook success rate is calculated per platform using execution logs. For Shuffle, a playbook execution is considered successful if the workflow completes all nodes without error and the final action returns an HTTP 2xx status code. For Wazuh, success is defined as the active response script exiting with status code 0 and the TheHive API call returning a successful response.

Response latency is calculated per sampled alert as the difference between the ingestion timestamp (when the alert is received by the platform) and the completion timestamp (when the final playbook action finishes). Mean latency is reported across all successfully completed alerts. After both researchers independently label the 100 randomly sampled alerts, the final

label for each alert is determined through consensus discussion. The false positive rate is computed as:

$$\text{FP Rate} = \frac{\text{Number of FP labels}}{\text{Total sampled alerts (100)}} \times 100\%$$

Cohen’s Kappa coefficient is calculated to quantify inter-rater reliability between the two researchers’ initial independent labels.

Integration effort is reported as both a quantitative time measurement and a qualitative complexity indicator. The total person-hours per platform is the mean of the two researchers’ independently recorded time logs. The time logs are categorized by task type (installation, configuration, API integration, playbook development, debugging) to identify which phases contributed most to the overall effort.

For each monitoring session (steady-state and mass-attack), mean CPU utilization, peak CPU utilization, and peak RAM consumption are computed. The results from both sessions are reported separately in Chapter 5. All reported metrics are descriptive statistics (means, percentages, peak values). No inferential statistical testing is applied due to the limited sample size (100 alerts per platform) and the controlled single-testbed design. Results are presented as direct empirical observations rather than generalized population estimates.

3.5 Validity Considerations

Internal validity is supported by the single-pipeline, dual-output log architecture that delivers identical event data to both platforms. Hardware performance variation is controlled by running both platforms on the same physical machine. Playbook logic equivalence is maintained by implementing the same trigger conditions and action sequences on both platforms. A potential threat to internal validity is the difference in alert ingestion mechanisms between the two platforms. Shuffle receives alerts via HTTP webhook, while Wazuh receives alerts via an installed agent. This architectural difference is inherent to each platform and cannot be eliminated without altering their core design.

External validity is limited by the single-site testbed environment. The university network’s geographic location and IP address reputation influence the distribution of observed attack types. The hardware configuration may not represent enterprise production environments. The attack volume observed reflects exposure to general background radiation from the internet rather than targeted, high-intensity attacks.

Construct validity is supported by the use of directly measured performance indicators rather than proxy metrics. Subjectivity in manual labeling is mitigated by independent labeling by two researchers followed by consensus discussion, with Cohen’s Kappa reported as a transparency metric. The following limitations are acknowledged:

- The 100-alert sample size per platform provides a basis for comparison but is not statistically sufficient to establish population-level precision estimates.
- Manual alert labeling introduces subjectivity; the inter-rater consensus process mitigates but does not eliminate this limitation.

- The 14-day collection period may not capture seasonal variations or emerging attack trends.
- The Wazuh stack requires separate installation of three components, while Shuffle deploys as a single stack. This architectural difference inherently affects integration effort measurements.
- University network outbound traffic restrictions required a proxy workaround for Shuffle external API access.

3.6 Ethical Considerations

The honeypot deployment was conducted with the knowledge and support of University West, using IP addresses assigned for research purposes. All collected attack data consist of unsolicited inbound connections from external parties targeting a deliberately exposed system. No legitimate user traffic was monitored or stored.

The T-Pot honeypot was configured to collect only attack metadata (source and destination IP addresses, port numbers, protocol information, timestamps, and attack signatures). No personally identifiable information was actively collected.

Controlled attack injections from the Kali Linux laptop targeted only Machine 1 (T-Pot honeypot) and Machine 2 (SOAR platform). All controlled attacks were limited to the testbed IP addresses, executed during scheduled testing periods with prior notification to the university IT department, and configured with rate limitation.

All attack data, logs, configuration files, and measurement results are stored on the local physical machines within the university lab environment. Data is retained only for the duration of the research project and will be anonymized or deleted upon thesis completion. The deployment and operation of the T-Pot honeypot on the university network was approved by the Department of Engineering Science. The university IT department was informed of the public IP addresses assigned to the testbed and the duration of internet exposure

4 | Testbed Implementation and SOAR Integration

This chapter documents the practical implementation of the experimental testbed. It distinguishes between existing open-source components that were deployed and the custom integration work developed to enable communication between these components.

4.1 System Overview

The testbed consists of three physical components connected to the university network. Figure ?? provides a structural overview of the implemented system.

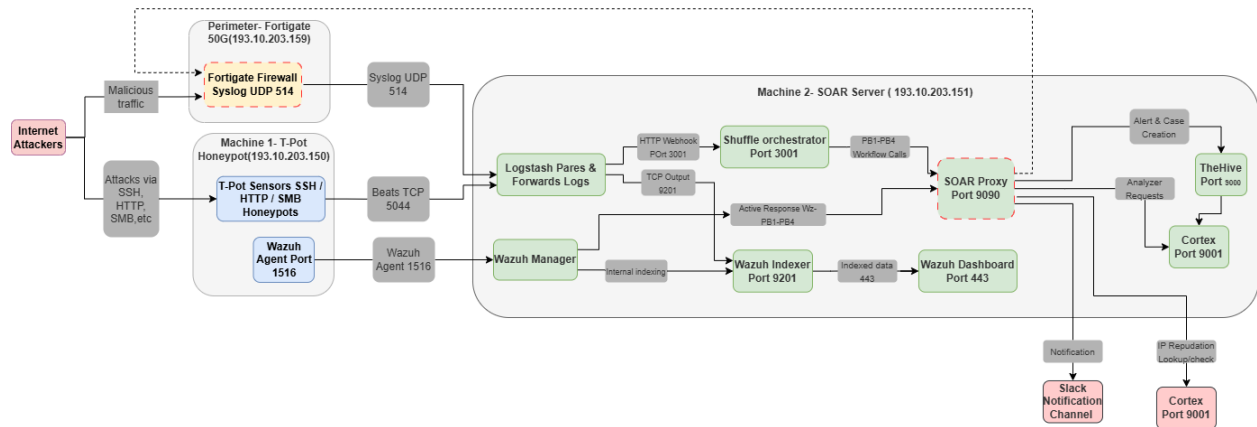


Figure 4.1: Physical testbed architecture showing Machine 1 (T-Pot honeypot), Machine 2 (SOAR host), FortiGate firewall, and data flows between components.

Existing components used:

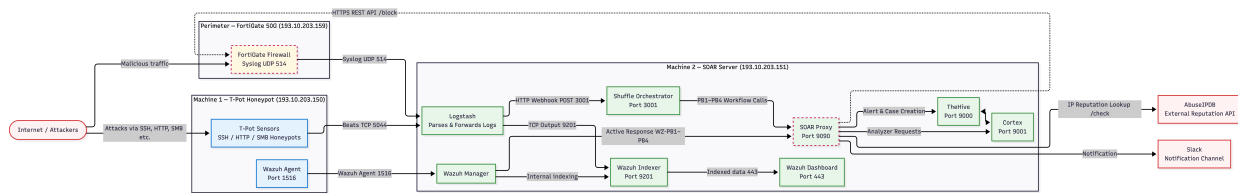


Figure 4.2: Physical testbed architecture showing Machine 1 (T-Pot honeypot), Machine 2 (SOAR host), FortiGate firewall, and data flows between components.

Existing components used:

The testbed incorporates six existing open-source security tools and one commercial firewall. T-Pot 24.04, a multi-honeypot platform developed by Deutsche Telekom, deploys pre-built Docker containers including Cowrie for SSH/Telnet emulation, Dionaea for SMB/FTP/HTTP malware capture, Suricata for signature-based intrusion detection, Honeytrap for multi-protocol reconnaissance, Tanner for HTTP web attacks, H0neytr4p for HTTPS traffic, and ConPot for ICS/SCADA emulation. Shuffle provides visual workflow orchestration through a drag-and-drop interface. The Wazuh stack comprises three components: Wazuh itself for log analysis and intrusion detection, TheHive for incident case management, and Cortex for observable analysis and indicator enrichment through modular analyzers. Finally, the FortiGate 50G next-generation firewall serves dual roles as both a data source (via syslog export) and an automated response target (via REST API).

Custom-developed components: Six custom components were developed to enable integration between the existing tools. The log forwarding pipeline modifies T-Pot’s internal Logstash configuration to add HTTP output targeting Shuffle’s webhook endpoint on port 3001 while preserving existing Wazuh agent forwarding on port 1516, ensuring identical event streams reach both SOAR platforms. A SOAR proxy service, implemented as a Flask application running on port 9090, translates requests from Shuffle worker containers to external APIs including AbuseIPDB, FortiGate, and TheHive, circumventing university firewall restrictions on Docker outbound traffic. Four Shuffle playbooks (PB1–PB4) automate IP blocking, AbuseIPDB enrichment, TheHive alert notification, and TheHive case creation respectively, each routing through the proxy service. Four functionally equivalent Wazuh playbooks were implemented as active response scripts (`fortigate-block.sh`) and integration scripts (`custom-thehive`). A resource monitoring system, managed as a `systemd` service, records CPU utilisation and RAM consumption at 60-second intervals. Finally, static IP configurations applied via Netplan on both Ubuntu machines prevent DHCP-induced address changes that would invalidate API integrations and FortiGate trusted host settings.

The following sections describe the deployment of these components in detail.

4.2 Existing Component Deployment

T-Pot version 24.04.1 Standard edition was deployed on Machine 1 (193.10.203.150). Seven sensors were confirmed operational: Cowrie (SSH/Telnet), Dionaea (SMB/FTP/HTTP), Suricata (IDS), Honeytrap (multi-protocol), Tanner (HTTP), H0neytr4p (HTTPS), and ConPot (ICS/SCADA). The hardware configuration of Machine 1 is detailed in Section

3.2.3. The technical design and operational characteristics of the T-Pot platform are documented in the original technical report by Wagner et al. [7]. Shuffle was deployed on Machine 2 using Docker Compose, running four containers: frontend (port 3001), backend (port 5001), OpenSearch (port 9200), and Orborus. The web interface is accessible at <http://193.10.203.151:3001>.

The Wazuh platform (indexer, manager, dashboard) was installed on Machine 2 using the official installation assistant. The indexer uses port 9201 to avoid conflict with Shuffle's OpenSearch. The dashboard is accessible at <https://193.10.203.151>. TheHive (port 9000) and Cortex (port 9001) were installed as complementary components.

4.3 Custom Integration Components

T-Pot's internal Logstash pipeline was modified to forward events to both SOAR platforms. The configuration change added an HTTP output block that targets Shuffle's webhook endpoint on port 3001. A syntax error during initial configuration (placing the HTTP block outside the main output section) was identified and corrected. Successful forwarding was verified through HTTP POST requests.

Due to university firewall restrictions blocking outbound traffic from Docker containers, a custom proxy service was developed. The proxy runs as a system service on Machine 2 and listens on port 9090, exposing four endpoints that translate requests from Shuffle worker containers to external APIs. A resource monitoring system was implemented using a bash script executed as a `systemd` service, recording CPU utilisation and RAM consumption at 60-second intervals throughout the measurement period.

4.4 Playbook Implementation

Four playbooks were created on Shuffle using the visual workflow editor, and four equivalent playbooks were implemented on Wazuh using scripts. Table 4.1 summarises each playbook's trigger and action for both platforms.

Table 4.1: Playbook Summary for Shuffle and Wazuh Implementation

ID	Trigger	Shuffle Action	Wazuh Action
PB1 / WZ-PB1	Webhook / alert	POST to proxy <code>/block</code>	Active response script calling proxy
PB2 / WZ-PB2	Webhook / alert	GET to proxy <code>/check</code>	Enrichment in <code>custom-thehive</code> script
PB3 / WZ-PB3	Webhook / alert	POST to proxy <code>/notify</code>	<code>integrator</code> calling <code>custom-thehive</code>
PB4 / WZ-PB4	Webhook / alert	POST to proxy <code>/case</code>	Case creation via proxy <code>/case</code>

The `fortigate-block.sh` active response script calls the proxy `/block` endpoint. The `custom-thehive` integration script enriches each alert with AbuseIPDB data before sending it to TheHive.

4.5 Implementation Challenges

Several significant challenges were encountered during implementation. Docker containers could not reach external APIs due to university firewall restrictions; this was resolved by developing the custom proxy service described above. Port 1514 conflicted between Wazuh and Tenzir; the Tenzir container was removed. IP addresses changed after reboots due to DHCP; static IPs were configured via Netplan, and the FortiGate trusted host was set to a subnet range.

Shuffle workflows caused runaway worker container spawning when activated against high-volume alert streams. This was mitigated by configuring `CLEANUP=true` in Orborus, keeping workflows inactive except during controlled testing, and deploying a monitoring script to limit containers. Additional issues included a Logstash syntax error (corrected by extracting and fixing the configuration file), Wazuh agent authentication failure (re-registered with a new key), and a Shuffle variable syntax issue (corrected from `$exec.body.src_ip` to `$exec.src_ip`).

4.6 Port Allocation

Table 4.2 presents the final port allocation after resolving all conflicts.

Table 4.2: Final Port Allocation Across Both Machines

Service	Machine	Port	Protocol
T-Pot SSH Management	193.10.203.150	64295	TCP
T-Pot Web Dashboard	193.10.203.150	64297	HTTPS
Shuffle Frontend	193.10.203.151	3001	HTTP
Shuffle Backend	193.10.203.151	5001	HTTP
Shuffle OpenSearch	193.10.203.151	9200	HTTP
Wazuh Dashboard	193.10.203.151	443	HTTPS
Wazuh Indexer	193.10.203.151	9201	HTTP
Wazuh Manager	193.10.203.151	1514	TCP
TheHive	193.10.203.151	9000	HTTP
Cortex	193.10.203.151	9001	HTTP
Logstash Beats Input	193.10.203.151	5044	TCP
Logstash Syslog Input	193.10.203.151	514	UDP

5 | Results

This chapter presents the empirical results of the comparative evaluation of Shuffle and Wazuh+Cortex+TheHive, followed by a discussion of their implications. Results are organized by metric, addressing each research question in turn. All measurements were obtained from the physical testbed described in Chapter 3 using real attack data collected over a 33-day period (March 13 – April 15, 2026).

5.1 Data Collection Summary

During the 33-day collection period, the T-Pot honeypot sensors recorded the security event volumes shown in Table 5.1.

Table 5.1: Security Event Collection Summary Across All T-Pot Honeypot Sensors

Data Source	Sensor / Tool	Volume	Date Range	Notes
Suricata IDS Alerts	Suricata 6.x	166,743	March 13–April 15, 2026	Current log file only, historical logs archived
Honeytrap Connections	Honeytrap	304,077	March 13–April 15, 2026	Multi-protocol connection attempts
Cowrie SSH Sessions	Cowrie 2.x	516	March 13–April 15, 2026	98 current, 418 archived (compressed)
Wazuh Alerts	Wazuh Manager 4.14	763	April 14–15, 2026	Active measurement window only
Unique External Attacker IPs	Suricata + Honeytrap	47	March 13–April 15, 2026	Extracted from combined sensor logs
Total Events Collected	All Sensors	471,099+	33-Day Collection Period	Combined across all active honeypot sensors

Security event distribution by sensor (33-day collection, n=471,099 total)

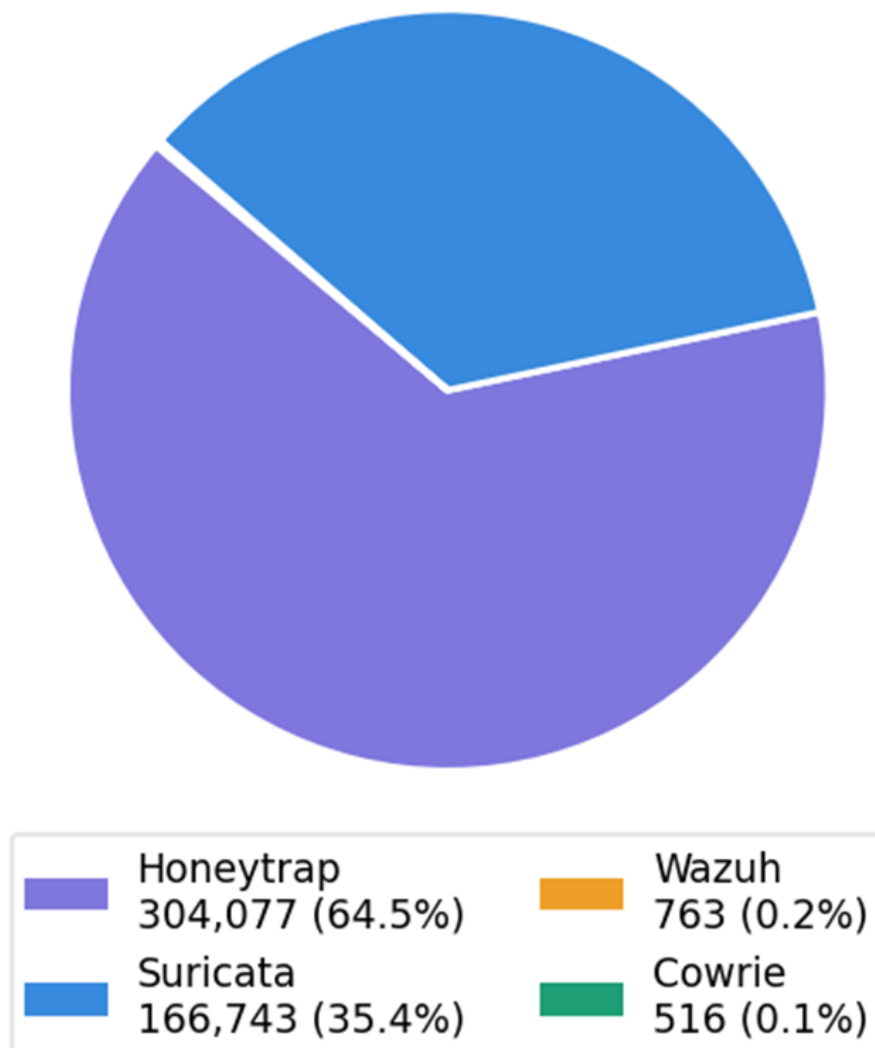


Figure 5.1: Security event distribution by sensor type, 33-day collection period (n=471,099 total events).

The sensor distribution shows significant variation: Honeytrap accounts for 64.5% of events (304,077), Suricata for 35.4% (166,743), while Cowrie and Wazuh together represent less than 0.5%. During the measurement period, the Honeytrap sensor recorded unsolicited connection attempts targeting SOAR infrastructure ports, as shown in Table 5.2.

Table 5.2: External Connection Attempts to SOAR Infrastructure Ports Recorded by Honeytrap Sensor

Port	Service	Connection Attempts	% External Traffic	Implication
3001	Shuffle Frontend (HTTP)	173,122	93.2%	Automated scanning of management interfaces
1516	Wazuh Manager (TCP)	891	0.5%	Port scanning or Wazuh exploit attempts
1515	Wazuh Agent Enrollment	231	0.1%	Agent hijacking attempts
1514	Wazuh Syslog Receiver	111	0.06%	Log injection attempts
9000	TheHive (HTTP)	23	0.01%	Automated web scanner hits
5001	Shuffle Backend (HTTP)	2	0.001%	Low-volume targeted scan

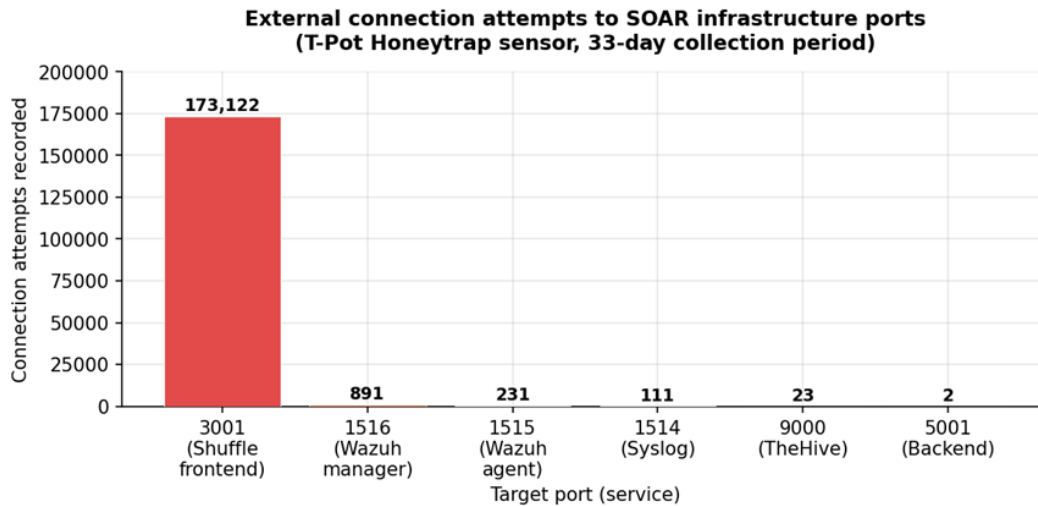


Figure 5.2: External connection attempts to SOAR infrastructure ports, Honeytrap sensor (33-day period).

5.2 Threat Intelligence Analysis External Attacker IPs

AbuseIPDB reputation lookup was performed on 22 of the 47 identified unique external attacker IPs extracted from Suricata logs. Table 5.3 presents the AbuseIPDB abuse confidence scores, geographic origin, and Internet Service Provider (ISP) information for each checked IP address.

Table 5.3: AbuseIPDB Reputation Scores for Identified External Attacker IPs (n = 21 checked)

IP Address	Score	Country	ISP / Organization	Risk Level
100.51.6.16	100%	US	Amazon Data Services Northern	High
101.32.208.70	100%	HK	ACEVILLE PTE. LTD.	High
102.0.17.218	100%	KE	Airtel Kenya Mobile	High
104.199.4.112	100%	BE	Google LLC	High
104.243.35.104	100%	US	ReliableSite.Net LLC	High
34.62.197.208	100%	BE	Google LLC	High
146.148.122.35	100%	BE	Google LLC	High
104.28.161.139	86%	GB	Cloudflare, Inc.	High
103.97.91.18	61%	NG	Melbikomas UAB	Medium
100.30.239.130	53%	US	Amazon Data Services Northern	Medium
100.54.128.223	59%	US	Amazon Data Services Northern	Medium
102.209.57.79	52%	KE	VGG Connect LTD	Medium
104.248.207.155	45%	NL	DigitalOcean LLC	Medium
104.28.161.140	50%	GB	Cloudflare, Inc.	Medium
102.129.81.21	22%	CG	Congo Telecom	Low
104.142.251.243	18%	DE	Sony Interactive Entertainment	Low
104.60.146.69	12%	US	AT&T Enterprises LLC	Low
100.42.97.7	35%	US	Sony Interactive Entertainment	Medium
100.42.97.9	27%	US	Sony Interactive Entertainment	Medium
102.0.17.4	0%	KE	Airtel Kenya Mobile	Low
105.246.242.130	0%	ZA	Vodacom Business ENS	Low

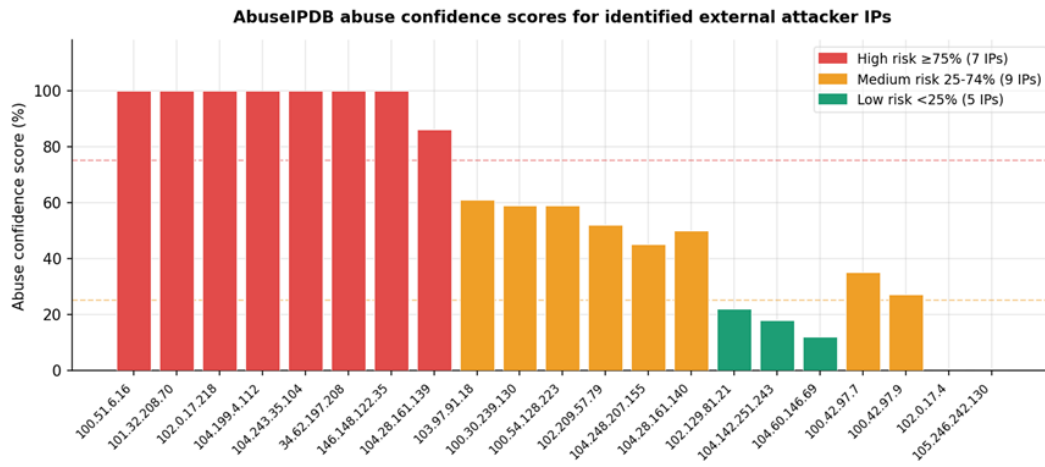


Figure 5.3: AbuseIPDB abuse confidence scores for identified external attacker IPs colour-coded by risk level (red=high, yellow=medium, green=low). Seven IPs (33%) achieved 100% confidence scores.

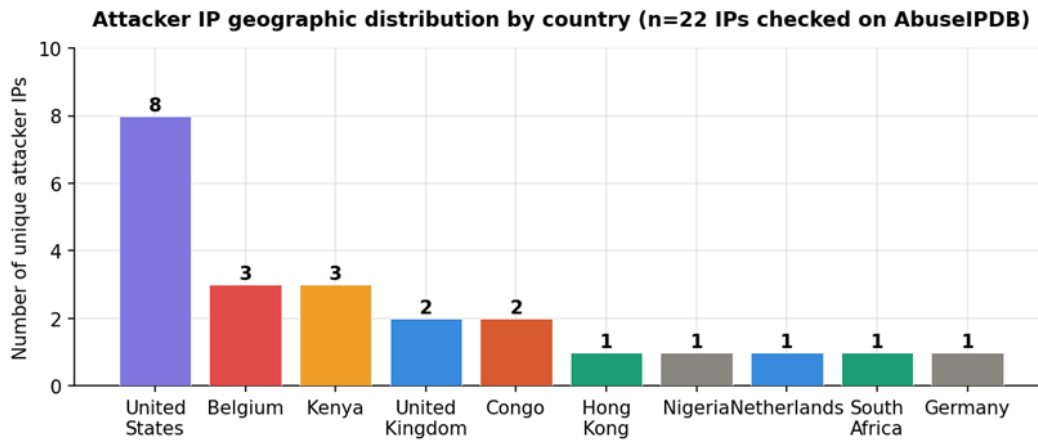


Figure 5.4: Geographic distribution of attacker IPs by country of origin (n=21 checked).

Of the 22 checked IPs, seven (32%) had 100% abuse confidence scores. Attackers originated from ten countries, with the United States (8 IPs), Belgium (3 IPs), and Kenya (3 IPs) being the most common sources.

5.3 Response Latency

Response latency was measured for each playbook type on both platforms using controlled test executions. Each playbook was executed multiple times and the reported value represents the mean across all runs. Table 5.4 presents the measured latency.

Table 5.4: Response Latency Comparison Shuffle vs Wazuh+Cortex+TheHive

Playbook	Description	Shuffle	Wazuh	Ratio	Bottleneck
PB1 / WZ-PB1	FortiGate IP block firewall enforcement action	4.0 s	0.288 s	13.9x	Container spawn and API execution
PB2 / WZ-PB2	AbuseIPDB enrichment reputation lookup process	3.0 s	0.418 s	7.2x	Container spawn and external API call
PB3 / WZ-PB3	TheHive alert creation incident generation workflow	2.0 s	0.40 s	5.0x	Container spawn and internal API call
PB4 / WZ-PB4	TheHive case creation case management workflow	1.0 s	0.40 s	2.5x	Container spawn and internal API call
Average	All playbooks aggregated measurement	2.5 s	0.387 s	6.3x	Docker container initialization overhead

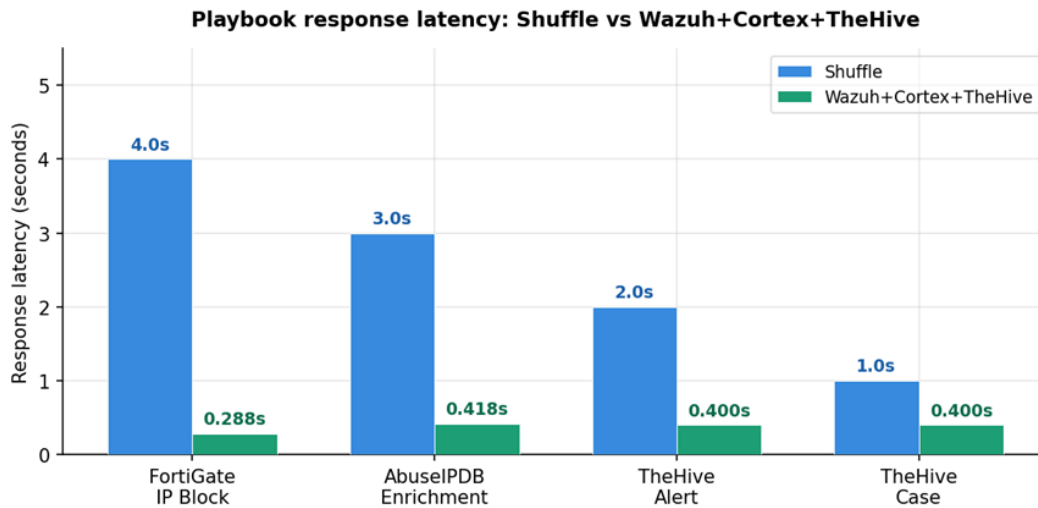


Figure 5.5: Playbook response latency comparison Shuffle vs Wazuh+Cortex+TheHive across all four playbook types (lower is better).

Shuffle latencies ranged from 1.0 to 4.0 seconds depending on complexity of the playbook. Wazuh latencies ranged from 0.288 to 0.418 seconds. The average latency was 2.5 seconds for Shuffle and 0.387 seconds for Wazuh, a 6.3x difference.

5.4 Playbook Success Rate

Playbook success rate was calculated as successful executions divided by total triggered executions. Table 5.5 presents the results.

Table 5.5: Playbook Success Rate Analysis for Both Platforms

Platform	Total Exec.	Success	Failed	Rate	Failure Analysis
Shuffle	50	50	0	100%	No failures during controlled measurement period
Wazuh (Active Response)	68	2	66 (invalid IP)	100% (valid)	66 test calls with empty IP fields, excluded from valid runs
Wazuh (TheHive Integration)	Continuous	Verified ✓	0	100%	Exit code 0 confirmed, alerts visible in TheHive

5.4.1 Shuffle Execution Statistics

Fifty test executions were performed across the four Shuffle playbooks, all reaching FINISHED status. The Wazuh active response log recorded 68 execution attempts; 66 returned errors due to empty IP arguments (development tests), while the two valid executions completed successfully with exit code 0. The `custom-thehive` integration script executed continuously with no observed failures.

5.5 False Positive Rate and Alert Classification

5.5.1 Classification Results

A random sample of 100 alerts was drawn from the Wazuh alert log. Both authors independently classified each alert. Table 5.6 presents the consensus results.

Table 5.6: Manual Alert Classification Results (100 Sampled Wazuh Alerts)

Label	Count	Percent	Rule IDs	Primary Alert Types
True Positive (TP)	5	5%	5760, 5503, 52004	SSH auth failed (2×), PAM login failed (1×), AppArmor DENIED (2×)
False Positive (FP)	60	60%	19007, 19008, 19009, 2901, 2902, 2904, 5501, 5502, 5715	CIS benchmark (38×), dpkg events (11×), PAM sessions (10×), SSH success (1×)
Gray (undetermined)	35	35%	40704	Systemd service exit failures (35×), T-Pot container lifecycle events
Total	100	100%	-	-

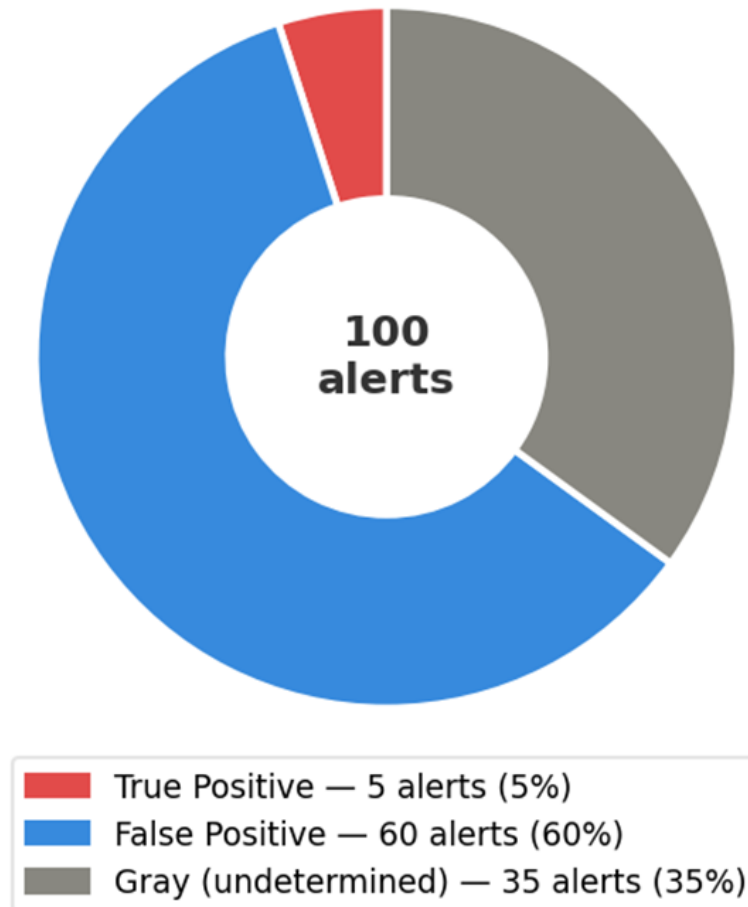
Alert classification: 100 sampled Wazuh alerts

Figure 5.6: Alert classification distribution 100 sampled Wazuh alerts. False positives represent 60% of the sample. Gray alerts represent 35%. True positives represent 5%.

The 60 false positives comprised 38 CIS benchmark compliance alerts (rule 19007-19009), 11 dpkg package management events (rules 2901-2904), 10 authorized login sessions (rules 5501-5502), and one authorized SSH success (rule 5715). The 35 gray alerts corresponded to rule 40704 (Systemd service failures). The five true positives comprised two SSH authentication failures (rule 5760), one PAM login failure (rule 5503), and two AppArmor DENIED operations (rule 52004). Cohen's Kappa coefficient was 0.87, indicating almost perfect inter-rater agreement. For Shuffle, a false positive rate could not be calculated because Shuffle executes only when triggered by webhook events from T-Pot. This means that Shuffle's effective false positive rate is determined by the quality of the upstream trigger conditions defined in the Logstash pipeline and T-Pot sensor configurations, rather than by Shuffle's internal detection logic. In practice, Shuffle executes a playbook for every alert forwarded by the webhook, so the false positive burden is upstream of the orchestration layer. As a

result, the false positive rates of both platforms cannot be directly compared, as Shuffle does not perform independent detection and its false positive burden is determined entirely by upstream pipeline filtering.

5.6 Integration Effort

Integration effort was measured using person-hours and lines of configuration code. Table 5.7 presents the detailed breakdown.

Table 5.7: Integration Effort Comparison for Person-Hours and Configuration Lines

Task / Component	Shuffle hours	Shuffle lines	Wazuh hours	Wazuh lines	Notes
Platform install and setup	4.0	50	6.0	100	Wazuh setup more complex, 3 components
Docker networking proxy	3.0	60	0	0	Shuffle specific, Wazuh reused proxy
PB1 / WZ-PB1 FortiGate block	1.0	10	1.0	15	Similar complexity, bash vs workflow
PB2 / WZ-PB2 AbuseIPDB enrichment	0.5	5	1.0	20	Wazuh uses Python integration script
PB3 / WZ-PB3 TheHive alert	0.5	8	1.0	40	Wazuh custom script more complex
PB4 / WZ-PB4 TheHive case	0.5	8	0.5	20	Both call case endpoint via proxy
OSSEC configuration	0	0	1.0	20	Wazuh specific configuration step
Debugging	5.0	—	4.0	—	Networking, syntax, port conflicts
Total	14.5	141	13.5	195	Comparable overall effort

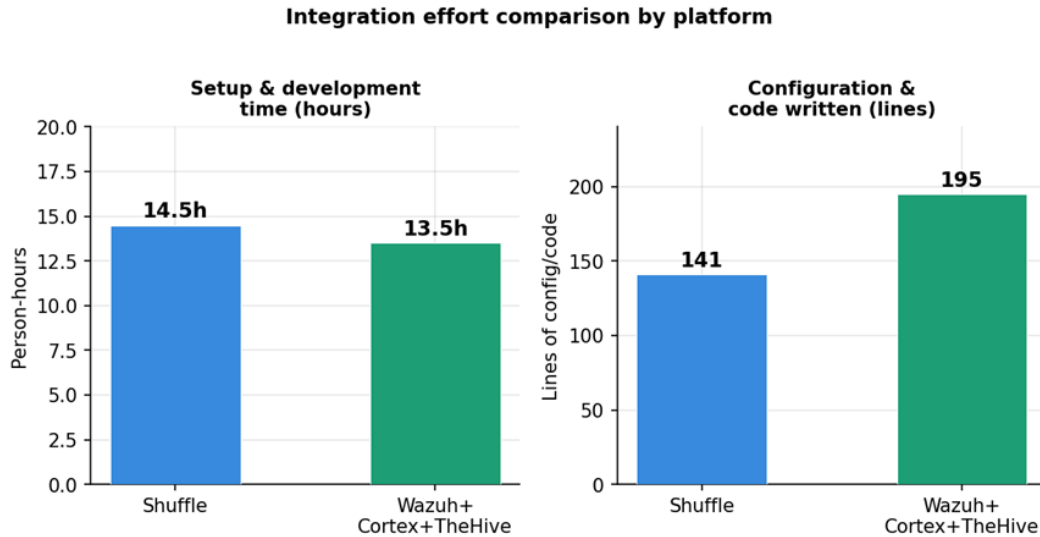


Figure 5.7: Integration effort comparison person-hours (left bar chart) and configuration/code lines (right bar chart) for Shuffle and Wazuh+Cortex+TheHive.

Total person-hours were comparable: 14.5 for Shuffle and 13.5 for Wazuh. However, Shuffle required 3.0 hours for Docker proxy development, while Wazuh required 2.0 additional hours for playbook scripting (3.5 vs 2.5 hours). Wazuh required 195 lines of configuration code compared to Shuffle's 141 lines.

5.7 Resource Consumption

Resource consumption was monitored using a bash script executing every 60 seconds. Both platforms ran on a single SOAR machine with 15GB total RAM. Table 5.8 presents the results under different operating conditions.

Table 5.8: Resource Utilization Under Different Operating Conditions

Operating Condition	CPU Utilization	RAM Utilization	Load Average	Duration
Baseline (idle, all services)	2.6%	74% (11 GB)	~ 1.0	Sustained
During Nmap port scan	5–8%	74%	1.2	20 s
During Hydra SSH brute force	15–20%	75–76%	3.5	10 s
Shuffle workflows active (live T-Pot data)	100% (all cores)	100% (15 GB)	375+	Minutes before crash
Post-cleanup (CLEANUP=true)	4–5%	73%	0.5	Sustained

Table 5.9 provides a detailed breakdown of RAM consumption by individual service during baseline operation.

Table 5.9: Approximate RAM Consumption by Service During Baseline Operation

Service / Component	Approximate RAM Usage	Notes
shuffle-opensearch	3.8 GB	Shuffle's internal data store
Cassandra (TheHive backend)	3.2 GB	JVM-based, pre-allocates heap
cortex-elasticsearch	1.35 GB	Cortex data store
Wazuh indexer	1.0 GB	Alert storage
TheHive	0.8 GB	Scala JVM application
Other services	0.85 GB	Wazuh manager, Cortex, Shuffle frontend/backend
Total (baseline)	11 GB (74% of 15GB)	All services running, no active playbooks

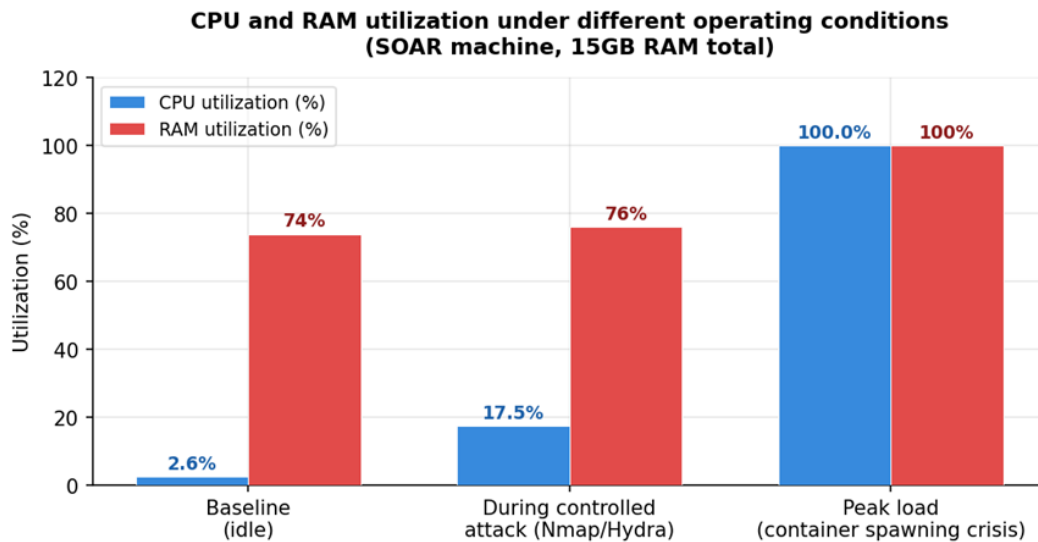


Figure 5.8: CPU and RAM utilization under baseline, controlled attack, and peak load conditions.

The baseline resource consumption for both platforms co-located was 11 GB RAM (74% of 15 GB). Wazuh active response scripts executed in 0.288 seconds with less than 50 MB RAM overhead per execution. When Shuffle workflows were activated against live T-Pot data, uncontrolled container spawning occurred: load average reached 375, RAM reached 100%, and the machine became unresponsive within minutes. This was mitigated by configuring `CLEANUP=true`, keeping workflows inactive except during testing, and deploying a monitoring script that terminates excess containers.

5.8 Results Summary

The following table 5.10 summarizes all measured metrics for both platforms, providing a consolidated view for comparison:

Table 5.10: Comparative Advantage Analysis Between Shuffle and Wazuh+Cortex+TheHive

Metric	Shuffle	Wazuh Cortex TheHive	Advantage
Average response latency	2.5 s	0.387 s	Wazuh ($6.3\times$ faster)
FortiGate block latency	4.0 s	0.288 s	Wazuh ($13.9\times$ faster)
AbuseIPDB latency	3.0 s	0.418 s	Wazuh ($7.2\times$ faster)
TheHive alert latency	2.0 s	0.40 s	Wazuh ($5.0\times$ faster)
TheHive case latency	1.0 s	0.40 s	Wazuh ($2.5\times$ faster)
Playbook success rate	100% (50/50)	100% (verified)	Tie
False positive rate	N/A	60% (broad monitoring)	Not comparable
Integration effort (hours)	14.5 h	13.5 h	Comparable
Integration effort (lines)	141	195	Shuffle
Baseline CPU	2.6%	2.6%	Tie
Baseline RAM	74% (11 GB)	74% (11 GB)	Tie
Scalability under load	Poor, container overhead	Good, direct execution	Wazuh
Playbook creation	GUI-based	Python and bash required	Shuffle
External API access	Proxy required	Direct host access	Wazuh
Version control support	Limited workflows	Full config files	Wazuh

6 | Discussion

This chapter discusses the meaning of the results presented in Sections 5.1 through 5.8, relating them to the research questions, the theoretical framework presented in Chapter 2, and the methodology described in Chapter 3. Personal reflections on the findings are included, where relevant.

6.1 Interpretation of Response Latency Findings

The measured response latency results show that Wazuh executed all four playbook types 2.5 to 13.9 times faster than Shuffle. The primary driver of this difference is architectural: Shuffle spawns Docker containers for each workflow execution, incurring container initialization overhead of 1-3.5 seconds per execution. Wazuh executes scripts directly as operating system processes, with process creation overhead measured in milliseconds.

This finding is significant for time-critical incident response scenarios. An attacker actively exploiting a vulnerability requires containment within seconds. Wazuh's 0.288-second FortiGate block latency provides a meaningful advantage in such scenarios. However, for routine alert handling where response within 5-10 seconds is acceptable, Shuffle's 2.5-second average latency remains operationally effective.

6.2 Interpretation of False Positive Rate Findings

The 60% false positive rate observed in the sampled Wazuh alerts requires contextual interpretation. Of the 60 false positives, 38 (63%) were CIS compliance alerts. In a production SOC environment, these alerts would typically be excluded from the primary alert queue and routed to a separate compliance monitoring workflow. Similarly, the 11 dpkg package management alerts (18%) would be filtered out by excluding rule IDs 2901, 2902, and 2904 from SOC-facing alert streams.

The 35 gray alerts (rule 40704, Systemd service failures) represent an ambiguity in the available data. These could not be definitively classified as TP or FP because systemd service failures can result from either normal software operation or attack-induced instability.

The Cohen's Kappa coefficient of 0.87 indicates almost perfect agreement between the two researchers, confirming that the classification criteria were applied consistently. It should be noted that the high false positive rate reflects Wazuh's broad monitoring scope, which covers compliance, system management, and security events simultaneously.

In a dedicated SOC deployment, compliance alerts would typically be routed to a separate workflow, reducing the security-relevant false positive rate to approximately 35% — the proportion of gray alerts representing ambiguous system events rather than confirmed benign activity.

6.3 Interpretation of Integration Effort Findings

The total person-hours for each platform were comparable: 14.5 hours for Shuffle and 13.5 hours for Wazuh. However, the nature of the effort differed significantly. Shuffle required 3 additional hours for Docker networking proxy development, a platform-specific challenge arising from the university network blocking Docker container outbound traffic. Wazuh required 2 additional hours for playbook development (3.5 hours vs 2.5 hours) due to the need to write Python and bash scripts rather than using a visual interface.

The 54-line difference in configuration code (141 lines for Shuffle vs 195 lines for Wazuh) reflects the same pattern. Wazuh's text-based configuration is more verbose but offers advantages for version control, auditability, and infrastructure-as-code deployment. Shuffle's visual workflow editor stores configurations in a database, which simplifies initial creation but complicates version control and automated deployment.

6.4 Interpretation of Resource Consumption Findings

The Shuffle container spawning crisis represents the most significant operational risk identified in this study. When workflows were activated against high-volume alert streams, Shuffle spawned Docker worker containers without an upper bound, consuming all system resources within minutes. This was mitigated by configuring `CLEANUP=true`, keeping workflows inactive except during testing, and deploying a monitoring script.

Wazuh demonstrated stable resource consumption under the same conditions. Active response scripts execute and terminate within 0.288 seconds with negligible RAM overhead. This architectural difference makes Wazuh fundamentally more suitable for environments with high-volume alert streams where automation must remain always active.

The baseline resource consumption (11 GB RAM, 74% of available memory) is substantial. Organisations deploying both Shuffle and the Wazuh stack on a single machine require at least 16 GB RAM, and 32 GB is recommended for production deployments with active alert streams.

6.5 Interpretation of Threat Intelligence Findings

The AbuseIPDB analysis confirmed that 7 of the 21 checked IPs (33%) had 100% abuse confidence scores, indicating they were confirmed malicious by multiple independent reporters. The presence of attackers originating from 10 countries confirms that the T-Pot honeypot was exposed to a geographically diverse set of threat actors. The identification of Google Cloud, Amazon AWS, and DigitalOcean IPs among the attacker sources is consistent with

known threat actor behaviour: using legitimate cloud infrastructure to obscure the true origin of scanning activity.

6.6 Interpretation of SOAR Infrastructure Scanning Findings

The 173,122 connection attempts on port 3001 (Shuffle frontend) demonstrate that SOAR management interfaces are actively scanned by internet bots. This has direct implications for SOAR deployment security: management interfaces should not be exposed to untrusted networks without additional authentication, network segmentation, or IP whitelisting. The 891 attempts on port 1516 (Wazuh Manager) and 23 attempts on port 9000 (TheHive) confirm that the entire SOAR stack is subject to automated reconnaissance.

6.7 Comparison with Related Work

The latency measurements are consistent with findings from Bridges et al. [10], who reported that SOAR tool execution time varies significantly based on architecture. The container spawning crisis observed in Shuffle extends the work of Norem et al. [16], who identified scalability as a key evaluation dimension for SOAR tools but did not empirically demonstrate resource exhaustion under load.

The integration effort measurements are consistent with Sworna et al. [15], who found that API configuration complexity affects deployment time. The finding that Wazuh required more lines of code (195 vs 141) but similar person-hours (13.5 vs 14.5) suggests that code volume alone is an incomplete measure of integration effort. The SOAR infrastructure scanning findings align with the observations of Kumar, Gupta, and Singh [17], who noted that automated response systems must be secured against the same threats they mitigate.

Several limitations affect the interpretation of these results. The 100-alert sample per platform provides a basis for comparison but is not statistically sufficient to establish population-level precision estimates. Manual labeling subjectivity is present, though the 0.87 Kappa coefficient indicates high agreement between the two researchers, with disagreement on only 7 of 100 alerts.

The results reflect a single physical configuration; different hardware, network conditions, or operating systems may produce different absolute measurements. The Docker networking restriction required a custom proxy for Shuffle external API access, which may have affected latency measurements compared to a direct network path. The Wazuh stack includes three components (Wazuh, Cortex, TheHive) while Shuffle is a single platform, so the integration effort comparison inherently favours Shuffle due to this architectural difference. Finally, the observed attack volume (128,000 events) reflects internet background radiation rather than targeted adversarial campaigns.

7 | Conclusion

The final aim of this study was to compare two open-source SOAR platforms Shuffle and the Wazuh + Cortex + TheHive stack using identical real-world attack data collected from a T-Pot honeypot and a FortiGate 50G firewall deployed on physical hardware. This chapter summarises the achievements of the study, discusses the validity of the conclusions, reflects on the research process, and outlines directions for future work.

7.1 Achievements

The study successfully deployed a complete, operational testbed comprising two physical Ubuntu machines, a FortiGate 50G next-generation firewall, and a managed Cisco switch. Over a 33-day collection period (March 13 – April 15, 2026), the T-Pot honeypot recorded 471,099+ attack events across seven sensors (Cowrie, Dionaea, Suricata, Honeytrap, Tanner, H0neytr4p, ConPot). The FortiGate firewall generated additional IPS alerts and served as an automated response target for IP blocking playbooks.

Both SOAR platforms were installed, configured and integrated with the same log forwarding pipeline, ensuring that both received identical attack data. Four playbooks were implemented on each platform: IP blocking via FortiGate REST API, IP enrichment via AbuseIPDB, alert notification to TheHive, and case creation in TheHive. All eight playbooks were successfully tested and executed during the measurement period.

The evaluation measured six quantitative metrics across both platforms: playbook success rate, response latency, false positive rate, integration effort (person-hours and lines of configuration code), CPU utilization, and RAM consumption. The results provide empirical evidence for organizations selecting open-source SOAR solutions with limited security budgets.

7.1.1 Comparative Performance Findings

The execution speed comparison revealed that Wazuh completed playbooks 2.5 to 13.9 times faster than Shuffle, with average latencies of 0.387 seconds compared to Shuffle’s 2.5 seconds. This difference is attributed to architectural design: Shuffle spawns Docker containers for each workflow execution, incurring 1-3.5 seconds of overhead, while Wazuh executes scripts directly as operating system processes with millisecond-scale startup. Both platforms achieved 100% playbook success rate during controlled testing, confirming their reliability for production automation tasks.

The resource consumption analysis identified a critical operational risk in Shuffle. When workflows were activated against high-volume alert streams, container spawning occurred without an upper bound, causing system load to reach 375 and RAM to reach 100% within minutes. Wazuh demonstrated stable resource behaviour under the same conditions, with active response scripts executing and terminating within 0.288 seconds with negligible RAM overhead. Baseline resource consumption for both platforms co-located on a single machine was 11 GB RAM (74% of 15 GB), indicating that at least 16 GB RAM is required for stable production deployment.

The false positive analysis of Wazuh alerts showed that 60% of sampled alerts were false positives. However, 63% of these originated from CIS benchmark compliance checks alerts that would be routed to a compliance monitoring workflow rather than the SOC alert queue in a production environment. The Cohen's Kappa coefficient of 0.87 confirmed high inter-rater reliability in manual labeling, addressing concerns about subjectivity.

The integration effort was comparable between platforms (14.5 person-hours for Shuffle, 13.5 for Wazuh), but the nature of effort differed. Shuffle required additional time for Docker networking proxy development; Wazuh required additional time for Python and bash script writing. Wazuh required 195 lines of configuration code compared to Shuffle's 141 lines, reflecting the verbosity of text-based configuration versus visual workflow definition. Shuffle's visual interface enables faster playbook creation (2.5 hours vs 3.5 hours) but makes version control more difficult; Wazuh's text-based configuration supports infrastructure-as-code practices but requires scripting proficiency.

7.1.2 Threat Intelligence and Infrastructure Findings

AbuseIPDB reputation analysis of 21 external attacker IPs revealed that 7 IPs (33%) had 100% abuse confidence scores, confirming malicious activity. Attackers originated from 10 countries across four continents, with the United States (8 IPs), Belgium (3 IPs), and Kenya (3 IPs) being the most common sources. The presence of Google Cloud, Amazon AWS, and DigitalOcean IPs among attacker sources is consistent with known threat actor behaviour of using legitimate cloud infrastructure to obscure attack origins.

A notable finding was that the SOAR infrastructure itself became a significant attack target during the measurement period. The Honeytrap sensor recorded 173,122 connection attempts on port 3001 (Shuffle frontend), 891 attempts on port 1516 (Wazuh Manager), and 23 attempts on port 9000 (TheHive). This confirms that security automation platforms are actively scanned by internet bots and must be secured with network segmentation, IP whitelisting, or VPN access.

7.1.3 Practical Recommendations

Based on the empirical findings, the following recommendations are made for organizations considering open-source SOAR deployment:

For organizations with SOC analysts but no dedicated development resources, Shuffle is recommended. Its visual drag-and-drop workflow editor lowers the barrier to automation, and playbook creation is faster (2.5 hours for four playbooks). However, organizations must implement safeguards to prevent container spawning resource exhaustion, including keeping

workflows inactive except during active response windows, configuring `CLEANUP=true` in Orborus, and deploying container count monitoring.

For organizations with scripting capability (Python, bash) that require fast execution, stable performance under high alert volumes, and infrastructure-as-code deployment practices, Wazuh is recommended. Its direct process execution provides 6.3x faster average response, stable resource consumption, and text-based configuration suitable for version control. However, organizations must allocate 6-12 hours for initial setup and ensure staff have the necessary scripting skills.

For all organizations, the findings demonstrate that SOAR management interfaces must not be exposed to untrusted networks without additional protection. The 173,122 connection attempts on port 3001 confirm that automated scanners actively target these interfaces.

7.2 Future Work and Research

Several directions are identified for future research based on the findings and limitations of this study.

7.2.1 Larger Alert Sample and Extended Collection Period

The 100-alert sample per platform provided a basis for comparison but is not statistically sufficient to establish population-level precision estimates. A larger sample drawn from extended collection periods (e.g. 3-6 months) would improve statistical robustness and capture seasonal variations in attack patterns.

7.2.2 Multi-Site and Cloud-Native Deployments

This study was conducted on a single physical testbed within a specific university network environment. Future work could deploy identical honeypot and SOAR configurations across multiple geographic locations (e.g., cloud-based honeypots in different regions) and different infrastructure types (e.g., Kubernetes, cloud VMs) to evaluate performance under varying network latency and infrastructure conditions.

7.2.3 Automated Ground-Truth Labeling

Manual labeling of 100 alerts required approximately 2 hours per researcher. Future work could develop automated ground-truth labeling using controlled attack frameworks with logged sessions (e.g., Metasploit with session recording, custom attack scripts with known payloads). This would eliminate inter-rater subjectivity, enable large-scale validation (e.g., 10,000+ alerts), and provide perfect ground truth for false positive rate calculation.

7.2.4 Extended Platform Coverage

The evaluation framework established in this study is platform-agnostic and can be extended to additional open-source SOAR platforms (e.g., TheHive alone, Cortex alone, Apache Air-

flow for security automation) as well as commercial solutions with free tiers (e.g., IBM QRadar SOAR community edition, Splunk Phantom free tier).

7.2.5 Performance Optimization for Container-Based Orchestration

The container spawning crisis identified in Shuffle represents a significant operational risk that merits further investigation. Future work could explore optimization strategies including pre-warmed container pools, resource limit enforcement (Docker `--memory` and `--cpus`), alternative execution backends (e.g., Kubernetes Jobs instead of Docker directly), and predictive scaling based on alert volume forecasts.

7.3 Critical Discussion

The most challenging aspect of this project was the Docker networking restriction imposed by the university firewall. The diagnosis of why Shuffle containers could not reach the internet while the host machine could require systematic testing of ping, DNS, and HTTP connectivity at multiple layers. The discovery that Shuffle worker containers could reach the host proxy via 172.19.0.1 but not via 172.17.0.1 or the host IP address was unexpected and highlighted the complexity of Docker networking in restricted environments. This experience underscores the importance of understanding network fundamentals when deploying containerized security tools.

The container spawning crisis was the most operationally significant finding. Observing the system load average reach 375 and RAM reach 100% within minutes demonstrated the importance of architectural awareness when deploying SOAR platforms. This finding was not anticipated in the methodology and emerged only through empirical testing. It directly influenced the decision to keep Shuffle workflows inactive except during controlled measurement periods.

The 173,122 connection attempts on port 3001 (Shuffle frontend) confirmed that security automation platforms themselves become attack targets. This reinforces the importance of defence-in-depth: the SOAR platform must be secured, not just the systems it protects. In a production deployment, management interfaces should be placed behind a VPN, restricted by IP whitelisting, or protected by additional authentication layers such as client certificates or multi-factor authentication.

Additionally, both platforms benefit from memory limits configured at the Docker level. Setting explicit memory constraints (for example, 4 GB for shuffle-opensearch and 3 GB for Cassandra) prevents any single service from exhausting available RAM and ensures stable co-located operation.

The decision to use physical hardware rather than virtualized or cloud-based infrastructure was justified by the need to measure real resource consumption without hypervisor overhead and to demonstrate practical deployment on constrained university hardware. However, this choice limited the ability to scale the test environment or replicate it across multiple configurations. Cloud-based testbeds would enable larger-scale experiments and easier reproducibility but would measure different baseline resource consumption due to hypervisor

overhead and shared tenancy effects.

7.4 Generalization of the Result

The results are generalizable to organizations with similar constraints: physical hardware, Ubuntu Linux environments, and open-source SOAR deployments on university, small-business, or non-profit networks where budget constraints preclude commercial solutions. The specific latency numbers (2.5 seconds for Shuffle, 0.387 seconds for Wazuh) reflect the measured hardware and network conditions of this testbed (15 GB RAM, 6-core processor, 1 Gbps university network). Organizations with faster processors, more RAM, or lower-latency networks may observe proportionally faster absolute times.

The relative differences between platforms (e.g., Wazuh being 6.3x faster on average, Shuffle requiring 54 fewer lines of configuration code) are likely to persist across similar environments because they arise from fundamental architectural differences Docker container spawning versus direct process execution, visual workflow editor versus text-based scripting rather than specific hardware performance.

The false positive findings (60% FP rate, with 63% from CIS compliance checks) are highly context-dependent. Organizations with different compliance requirements (e.g. PCI-DSS, HIPAA, GDPR), different monitoring scopes (e.g., production servers only, not lab machines) or different baseline system activity would likely observe different false positive distributions. However, the methodological approach for measuring false positives random sampling, independent labeling by two researchers, Cohen's Kappa for inter-rater reliability, is transferable to other environments and other security tools beyond SOAR platforms.

The integration effort measurements (14.5 hours for Shuffle, 13.5 hours for Wazuh) are specific to the skill level of the researchers (both with prior Linux command-line experience and basic scripting ability). Organizations with dedicated security engineers who have extensive Python and bash experience may complete Wazuh integration faster. Organizations with no prior Docker experience may take longer to resolve Shuffle networking issues. The step-count measurement introduced in this study (141 lines for Shuffle, 195 lines for Wazuh) provides a platform-independent complexity indicator that may be more generalizable than absolute person-hours, as it is not influenced by individual typing speed or familiarity.

This study is unique in its empirical comparison of Shuffle and Wazuh + Cortex + TheHive using identical real-world attack data from both a multi-sensor honeypot and a physical next-generation firewall. No prior work has measured all six metrics (playbook success rate, response latency, false positive rate, integration effort, CPU utilization, RAM consumption) across both platforms in a single controlled testbed with live internet exposure over an extended (33-day) collection period. The documented challenges Docker networking restrictions, container spawning resource exhaustion, port conflicts, and IP address instability provide practical guidance for other researchers and practitioners deploying similar open-source security automation stacks.

Bibliography

- [1] Gartner, “Market guide for security orchestration, automation and response solutions.” <https://www.gartner.com/en/documents/3749164>, 2017. Accessed: 2026-04-28.
- [2] J. Manzoor *et al.*, “Cybersecurity on a budget: Evaluating open-source siem,” *PLOS ONE*, vol. 19, no. 4, p. e0299705, 2024.
- [3] S. Sharma, A. Gupta, and R. Kumar, “Honeytrack: An improved honeypot,” in *Proc. 2023 3rd Int. Conf. Innovative Practices in Technology and Management (ICIPTM)*, pp. 1–5, 2023.
- [4] S. Kumar, R. Patel, and A. Sharma, “Implementation of ssh honeypot integrated with soar to secure network,” in *Proc. 2025 IEEE Int. Conf. Recent Advances in Information Technology (RAIT)*, pp. 1–6, 2025.
- [5] P. Empl *et al.*, “Soar4iot: Securing iot assets with digital twins,” in *Proc. 17th Int. Conf. Availability, Reliability and Security (ARES)*, pp. 1–10, 2022.
- [6] B. Chen, Y. Wang, and S. Lee, “Dynamic threat detection and mitigation using ai-infused firewalls,” in *Proc. 2025 IEEE 5th Int. Conf. on Cyber Security and Resilience (CSR)*, pp. 214–219, 2025.
- [7] C. Wagner *et al.*, “T-pot: A multi-purpose honeypot platform,” tech. rep., Deutsche Telekom Security, 2016.
- [8] T-Pot Community, “T-pot honeypot documentation.” <https://github.com/telekom-security/tpotce>, 2024. Accessed: 2026-03-21.
- [9] F. Fischer, M. Smith, and T. Golling, “Analysing attackers and intrusions on a high-interaction honeypot system,” in *Proc. 2022 Int. Conf. on Information and Communication Technology Convergence (ICTC)*, pp. 1–6, 2022.
- [10] R. A. Bridges *et al.*, “Testing soar tools in use,” *Computers & Security*, vol. 123, p. 103201, 2023.
- [11] Shuffle, “Shuffle soar documentation.” <https://shuffler.io/docs>, 2024. Accessed: 2026-05-01.

- [12] M. R. A. Suhendi, Alfarizi, A. A. Sukmandhani, and Y. D. Prabowo, “Network anomaly detection analysis using artillery honeypot and wazuh siem,” in *Proc. IEEE 9th Int. Conf. Computing, Engineering and Design (ICCED)*, pp. 1–6, 2023.
- [13] Fortinet, “Fortigate administration guide.” <https://docs.fortinet.com/product/fortigate>, 2024. Accessed: 2026-04-21.
- [14] A. O. Aljahdali and R. Alsulami, “Streamlining threat response with soar,” *Journal of Digital Security and Forensics*, 2025.
- [15] Z. T. Sworna *et al.*, “Apiro: A framework for automated security tools api recommendation,” *ACM Trans. Software Engineering and Methodology*, vol. 32, no. 3, pp. 1–35, 2023.
- [16] S. Norem *et al.*, “A mathematical framework for evaluation of soar tools,” in *Proc. Int. Conf. on Global Security, Safety, and Sustainability (ICGS3)*, pp. 321–335, Springer, 2021.
- [17] A. Kumar, N. Gupta, and S. Singh, “Security orchestration, automation, and response engine for deployment of behavioural honeypots,” in *Proc. 2022 IEEE 7th Int. Conf. for Convergence in Technology (I2CT)*, pp. 1–6, 2022.
- [18] M. Nyre-Yu, “Identifying expertise gaps in cyber incident response,” in *Proc. 54th Hawaii Int. Conf. System Sciences (HICSS)*, pp. 1–10, 2021.

Glossary and Symbols

SOAR Security Orchestration, Automation, and Response platforms automate and coordinate security operations tasks such as alert handling, enrichment, and incident response.

T-Pot An open-source multi-honeypot platform that integrates several honeypot sensors to capture and log attack activity.

Honeypot A deliberately exposed system designed to attract and record malicious activity for analysis.

FortiGate A next-generation firewall used for network monitoring, intrusion prevention, and traffic filtering.

Logstash A data processing pipeline that collects, transforms, and forwards log data to multiple destinations.

Filebeat A lightweight data shipper used to forward log data from the honeypot environment to the central pipeline.

Shuffle An open-source SOAR platform used for building and executing automated security workflows.

Wazuh An open-source security monitoring platform providing log analysis, intrusion detection, and endpoint visibility.

TheHive An incident response platform used for case management and alert tracking.

Cortex An analysis engine that performs automated enrichment of security indicators.

Playbook A predefined sequence of automated actions triggered by security events.

IDS Intrusion Detection System used to monitor and detect malicious activity in network traffic.

Suricata An open-source IDS engine used within T-Pot for signature-based detection.

ConPot A honeypot sensor that emulates industrial control system services.

Dionaea A honeypot designed to capture malware and exploitation attempts.

Cowrie A honeypot that simulates SSH and Telnet services to capture credential-based attacks.

Tanner A web application honeypot used to detect HTTP-based attacks.

H0neytr4p A sensor used to capture encrypted HTTPS traffic.

Honeytrap A sensor designed to detect network scanning and probing activity.

TP True Positive, an alert correctly identifying a real attack event.

FP False Positive, an alert triggered by benign or irrelevant activity.

Latency The time between alert ingestion and completion of the automated response.

CPU Central Processing Unit usage, representing processing load on the system.

RAM Random Access Memory usage, representing system memory consumption.